

Projeto de Desenvolvimento do Site LCQUI

Planejamento Funcional, Arquitetural, de Dados e de Implementação

Zadoque Carneiro

4 de setembro de 2026

Sumário

1 Introdução

O LCQUI é o Laboratório de Ciências Químicas da Universidade Estadual do Norte Fluminense Darcy Ribeiro (UENF), vinculado ao Centro de Ciência e Tecnologia (CCT). Ele funciona como uma unidade de ensino, pesquisa e suporte técnico voltada para a área de química.

2 Como vai ser o Sistema

Criar um sistema online que contenha as informações dos bens patrimoniais, reagentes e materiais do almoxarifado do setor de Graduação do LCQUI, bem como criar uma plataforma geral de acesso aos roteiros das aulas experimentais desenvolvidas nos laboratórios desse setor. A intenção é, dessa forma, auxiliar os professores do LCQUI na consulta dos itens existentes, registro de novos itens com maior agilidade, colaborando na identificação de itens inservíveis, em escassez ou que necessitam de reposição ou reparos, auxiliando assim na gestão da chefia do LCQUI. Em relação aos roteiros experimentais, os professores poderão utilizar a plataforma para lançamento ou troca de arquivos e divulgação junto aos alunos.

3 Stakeholders

i Importante

Um mesmo usuário pode possuir múltiplos papéis, observadas as combinações permitidas na Matriz de Papéis e Permissões da seção de Regras de Negócio. O papel *Chefe Geral* é mutuamente exclusivo com os demais papéis para uma mesma conta. O papel *Bolsista* possui como pré-condição existencial e estrutural o papel de *Aluno* ($Bolsista \implies Aluno$).

3.1 Chefe Geral

Esse é o maior cargo do sistema: Esse cargo gerencia os outros cargos.

- Poder cadastrar outros usuários e definir os papéis deles no sistema, de acordo com as regras descritas em *Regras de Negócio* e na *Descrição das telas*.
- Pode cadastrar um Almoxarifado.
- Ao cadastrar um Gestor de Almoxarifado, pode colocá-lo responsável por mais de um almoxarifado.

- Ao cadastrar um Gestor de Almoxarifado, pode vinculá-lo imediatamente para gerenciar um ou mais almoxarifados, ou apenas cadastrar o usuário (deixando a vinculação para um momento futuro, resultando em uma dashboard sem almoxarifados ativos inicialmente).
- Atribuir o papel de Bolsista a um usuário que já possua o papel Aluno.

3.2 Gestor de Almoxarifado

Pode haver mais de um gestor, ele(s) pode(m):

- Gerar um pdf de etiquetas de frasco
- Adicionar/alterar a quantidade de um reagente no estoque.
- Pesquisar por um reagente no estoque.
- Cadastrar uma nova especificação de reagente, informando os dados obrigatórios definidos pelo tipo de substância e pelo estado físico.
- Marcar um reagente como removido (não exclui do banco, apenas muda o status).
- Alterar o status de um reagente: marcar empréstimo, devolução (através de pesagem), descarte, vazio ou quebrado.
- Autorizar expressamente o uso de frascos vencidos para fins exclusivamente didáticos e demonstrações.
- Gerar relatório mensal (ou de um período específico) para um almoxarifado ou para todos.
- Editar/adicionar propriedades de um reagente.

3.3 Gestor de Bens Patrimoniais

Pode haver mais de um gestor de bem patrimonial, ele(s) pode(m):

- Visualizar as requisições dos professores para adicionar/editar um bem patrimonial.
- Quando a requisição do professor é para colocar um bem como inservível e o gestor aprova, o gestor deve dar baixa do bem patrimonial através do processo da UENF (abrir processo no SEI etc.). Após esse processo, ele marca no sistema que foi dada baixa.
- Adicionar um bem patrimonial e criar resumos de catálogo.

- Editar um bem patrimonial e suas propriedades.
- Gerar e baixar relatório de bens patrimoniais mensal (ou de período específico).
- Visualizar os bens patrimoniais existentes.
- Pesquisar bens patrimoniais por número de patrimônio ou nome.
- Filtrar bens patrimoniais por local (prédio, andar e sala), estado de conservação ou status.
- Imprimir Relatório mensal (ou período personalizado), ou sobre um Bem patrimonial específico podendo filtrar por prédio, andar, sala, Nome do responsável, etc.

3.4 Professor

Com certeza haverá mais de um, eles podem:

- Criar uma turma (especificando nome e capacidade máxima de alunos).
- Copiar o código de uma turma dele.
- Criar um post em uma turma (ao criar um post, o professor pode selecionar um roteiro que ele já tenha feito o upload, um roteiro que outro professor compartilhou com ele, ou ainda fazer upload de um roteiro do próprio computador – isso abre um modal para cadastrar o roteiro).
- Adicionar usuários apenas como alunos por e-mail (mandando o link para eles trocarem a senha).
- Pesquisar por bens patrimoniais.
- Abrir uma requisição de adição ou edição de bem patrimonial (analisada por um gestor de bens patrimoniais).
- Adicionar novos Locais (Prédio, Andar e Sala) no sistema de forma autônoma (por exemplo, ao fazer uma requisição, selecionar a opção "Novo Local" e registrá-lo).
- Solicitar/receber empréstimo de reagentes do almoxarifado; o registro operacional da retirada é efetuado pelo Gestor de Almoxarifado.
- Adicionar um roteiro de experimento (formato PDF).
- Vincular um roteiro de experimento a uma turma.

- Compartilhar um roteiro de experimento com outros professores.
- Acessar a aba de roteiros compartilhados (dividida em: “Compartilhados comigo” e “Eu compartilhei”).
- Baixar um roteiro (seja adicionado por ele mesmo ou compartilhado com ele por outro professor).
- Vincular um roteiro de experimento compartilhado com ele por outro professor a uma turma dele.
- Arquivar e desarquivar turmas sob sua responsabilidade.

3.5 Aluno

Esses com certeza serão maioria. Eles podem:

- Entrar em uma turma usando o código disponibilizado pelo professor (respeitado o limite de capacidade da turma).
- Visualizar suas turmas e o professor respectivo de cada turma.
- Baixar qualquer roteiro de qualquer turma da qual façam parte.
- Visualizar os colegas que estão na mesma turma.
- Comentar em um post (o comentário é visível tanto para o professor quanto para os colegas).

3.6 Bolsista

Seja de graduação ou pós-graduação, este papel identifica um usuário que possui o papel de Aluno e uma autorização adicional para retirada de reagentes (*Bolsista* $\Rightarrow$ *Aluno*). Ele é mutuamente compatível com Aluno, mas possui restrições próprias para acumulação com papéis de gestor.

- Ele pode ver os reagentes e seus estoques assim como os professores.
- O gestor de reagentes pode registrar retiradas e devoluções para esses usuários.
- É mutuamente exclusivo com o papel de Gestor de Almoxarifado para uma mesma conta, em virtude do princípio de Segregação de Funções (SoD).

3.7 Matriz de Papéis e Permissões

Para eliminar ambiguidades de autorização, a regra de acesso é centralizada na matriz abaixo. As permissões são do papel ativo do usuário e nunca são determinadas por um papel informado pelo cliente.

Ação	Chefe	Gestor Al- mox.	Gestor Patr.	Professor	Aluno	Bolsista
Gerenciar usuários e papéis	✓	–	–	–	–	–
Gerenciar almoxarifados	✓	–	–	–	–	–
Consultar estoque de reagentes	✓	✓	–	✓	✓	✓
Cadastrar/editar reagente e frasco	✓	✓	–	–	–	–
Registrar retirada/devolução	✓	✓	–	retirante	–	retirante
Gerenciar bens patrimoniais	✓	–	✓	leitura	–	–
Abrir requisição de patrimônio	✓	–	–	✓	–	–
Responder requisição de patrimônio	✓	–	✓	–	–	–
Gerenciar turmas próprias	✓	–	–	✓	–	–
Ingressar em turma	–	–	–	–	✓	✓
Criar/comentar posts permitidos	–	–	–	✓	✓	✓
Gerenciar roteiros próprios	✓	–	–	✓	–	–
Compartilhar roteiros	✓	–	–	✓	–	–
Cadastrar/editar matérias	✓	–	–	✓	–	–
Gerar etiquetas de frasco	✓	✓	–	–	–	–

i Regra de autorização

A Cloud Function responsável por cada mutação deve resolver os papéis a partir dos registros persistidos e, quando aplicável, validar também o escopo do recurso. Por exemplo, um Gestor de Almoxarifado só atua sobre almoxarifados aos quais está vinculado. O frontend pode ocultar ou desabilitar ações, mas não constitui autorização.

4 Modelagem Entidades SQL - 3FN

i Como ler as caixas de entidade

Cada caixa a seguir representa uma **tabela do PostgreSQL** na Terceira Forma Normal (3FN). Todos os atributos são **NOT NULL** por padrão, salvo quando explicitamente indicado `NULL`. O nome do campo aparece à esquerda e o **tipo de dado** sugerido para o PostgreSQL aparece à direita, em cinza. Quando aplicável, um selo adicional indica **PK** (chave primária), **FK** (chave estrangeira) ou uma restrição como `NULL` (aceita valor nulo) ou `UNIQUE`.

4.1 Usuario

Usuario

<i>id</i>	SERIAL	PK
<i>nome</i>	VARCHAR(150)	NOT NULL
<i>email</i>	VARCHAR(150)	UNIQUE, NOT NULL
<i>profile_url_photo</i>	TEXT	NULL

4.2 Chefe Geral

Chefe_Geral

<i>id_usuario</i>	INTEGER	PK, FK
-------------------	---------	--------

4.3 Gestor de Almoxarifado

Gestor_Almoxarifado

<i>id_usuario</i>	INTEGER	PK, FK
-------------------	---------	--------

4.4 Tabela Gestor de Almoxarifado × Almoxarifado (N para N)

Gestor_Almoxarifado_x_Almoxarifado

id_gestor_almoxarifado

INTEGER PK, FK

id_almoxarifado

INTEGER PK, FK

Regra de negócio

A chave primária composta (*id_gestor_almoxarifado*, *id_almoxarifado*) impede vincular o mesmo gestor duas vezes ao mesmo almoxarifado.

4.5 Gestor de bens patrimoniais

Gestor_Bens_Patrimoniais

id_usuario

INTEGER PK, FK

4.6 Bem patrimonial

Bem_Patrimonial

id

SERIAL PK

id_resumo_bem_patrimonial

INTEGER FK, NOT NULL

numero_patrimonio

VARCHAR(30) UNIQUE, NOT NULL

estado_conservacao

ENUM BOM, REGULAR, RUIM, NOT NULL

id_local

INTEGER FK, NOT NULL

photo_url

TEXT NOT NULL

documento_dado_baixa_pdf_url

TEXT NULL

nome_responsavel_sei

VARCHAR(150) NOT NULL

status

ENUM Ativo, Inservivel, Ja_dado_baixa, NOT NULL

descricao_complementar

VARCHAR(200) NULL

i Observação

id_resumo_bem_patrimonial, *numero_patrimonio*, *estado_conservacao*, *id_local*, *photo_url*, *nome_responsavel_sei* e *status* são estritamente **NOT NULL**. Apenas *documento_dado_baixa_pdf_url* admite NULL (preenchido exclusivamente quando o bem atinge o status *Ja_dado_baixa*) e *descricao_complementar* (opcional para notas individuais daquela unidade física).

4.7 Resumo bem patrimonial**Resumo_Bem_Patrimonial***id*

SERIAL

PK

nome

VARCHAR(150)

NOT NULL

descricao

TEXT

NOT NULL

i Regra de Identidade de Bens Patrimoniais

Resumo_Bem_Patrimonial representa o modelo catalográfico do equipamento (ex.: “Microscópio Óptico Binocular Coleman”). Vários bens físicos compartilham o mesmo resumo. A alteração do nome do resumo é restrita ao Gestor de Bens e reflete em todos os itens do modelo. Caso um professor solicite alteração de nome para um bem individual, a operação representa reclassificação para outro resumo existente ou criação de um novo modelo, preservando a integridade dos demais equipamentos.

4.8 Historico bem patrimonial

Historico_Bem_Patrimonial

<i>id</i>	SERIAL	PK
<i>id_bem_patrimonial</i>	INTEGER	FK, NOT NULL
<i>timestamp</i>	TIMESTAMP	NOT NULL
<i>tipo</i>	ENUM	cadastro, edicao, baixa, NOT NULL
<i>id_usuario</i>	INTEGER	FK, NOT NULL

4.9 Alteracao bem patrimonial

Alteracao_Bem_Patrimonial

<i>id</i>	SERIAL	PK
<i>id_historico</i>	INTEGER	FK, NOT NULL
<i>campo</i>	VARCHAR(60)	NOT NULL
<i>valor_anterior</i>	TEXT	NOT NULL
<i>valor_novo</i>	TEXT	NOT NULL

4.10 Requisicao Professor edicao Bem patrimonial

Requisicao_Edicao_Bem_Patrimonial

<i>id</i>	SERIAL	PK
<i>id_bem_patrimonial</i>	INTEGER	FK, NOT NULL
<i>status</i>	ENUM	pendente, aprovada, rejeitada, NOT NULL
<i>feita_em</i>	TIMESTAMP	NOT NULL
<i>respondida_em</i>	TIMESTAMP	NULL
<i>id_usuario_solicitante</i>	INTEGER	FK, NOT NULL
<i>id_usuario_respondente</i>	INTEGER	FK, NULL
<i>novo_nome</i>	VARCHAR(150)	NULL
<i>novo_status</i>	ENUM	Ativo, Inservivel, Ja_dado_baixa, NULL
<i>novo_estado_conservacao</i>	ENUM	BOM, REGULAR, RUIM, NULL
<i>novo_id_local</i>	INTEGER	FK, NULL
<i>nova_photo_url</i>	TEXT	NULL
<i>motivo</i>	TEXT	NOT NULL
<i>justificativa_resposta</i>	TEXT	NULL

Regra de negócio

RF10: *UNIQUE(id_bem_patrimonial) WHERE status = 'pendente'* – impede mais de uma requisição de edição pendente simultânea para o mesmo bem patrimonial. Um índice único parcial resolve isso no PostgreSQL; no Firestore, a checagem precisa ser feita na própria *Cloud Function* de criação, dentro de uma transação (ver seção *Tecnologia e Relatórios*).

4.11 Requisicao Professor adicao Bem patrimonial

Requisicao_Adicao_Bem_Patrimonial

<i>id</i>	SERIAL	PK
<i>numero_patrimonio_proposto</i>	VARCHAR(30)	NOT NULL
<i>status</i>	ENUM	pendente, aprovada, rejeitada, NOT NULL
<i>feita_em</i>	TIMESTAMP	NOT NULL
<i>id_bem_patrimonial_se_aprovado</i>	INTEGER	FK, NULL
<i>respondida_em</i>	TIMESTAMP	NULL
<i>id_usuario_solicitante</i>	INTEGER	FK, NOT NULL
<i>id_usuario_respondente</i>	INTEGER	FK, NULL
<i>estado_conservacao_proposto</i>	ENUM	BOM, REGULAR, RUIM, NOT NULL
<i>photo_url_proposta</i>	TEXT	NULL
<i>nome_responsavel_proposto</i>	VARCHAR(150)	NOT NULL
<i>id_local</i>	INTEGER	FK, NOT NULL
<i>id_resumo_bem_patrimonial</i>	INTEGER	FK, NULL
<i>nome_resumo_proposto</i>	VARCHAR(150)	NULL
<i>descricao_resumo_proposta</i>	TEXT	NULL
<i>motivo</i>	TEXT	NOT NULL

i Observação

Se o local desejado não existir no banco, o professor deve primeiro criar o *Local* no sistema, para então vincular a respectiva FK (*id_local*) nesta requisição.

Regra de negócio

RF10b: *UNIQUE(numero_patrimonio_proposto) WHERE status = 'pendente'* – impede mais de uma requisição de adição pendente simultânea para o mesmo número de patrimônio. No PostgreSQL, isso é implementado como índice único parcial; no Firestore, a verificação é reimplementada na Cloud Function de criação da requisição, dentro de uma transação.

4.12 Lock de Requisição de Patrimônio

Locks_Requisicao_Patrimonio

id (docId determinístico)

VARCHAR(100) **PK**

criado_em

TIMESTAMP NOT NULL

Lock Determinístico para Unicidade de Requisição Pendente

O Firestore não adquire trava pessimista sobre **ausência** de documento em uma query transacional. Por isso, a garantia de que não existam duas requisições pendentes para o mesmo bem ou plaqueta **não pode** ser implementada pela consulta `WHERE status = 'pendente'` dentro de uma transação – dois clientes simultâneos lerão conjunto vazio, ambos passarão na validação e ambos gravarão, violando RF10/RF10b.

A solução é o padrão *deterministic lock*: a Cloud Function tenta criar um documento cujo **docId** é **derivado deterministicamente** do recurso disputado. O Firestore garante que apenas uma gravação com o mesmo docId vence quando executadas concorrentemente dentro de uma transação.

- Para requisição de **edição**: `docId = bem_edicao_{id_bem_patrimonial}`
- Para requisição de **adição**: `docId = bem_adicao_{numero_patrimonio_proposto}`

O lock é **removido** pela função `responderRequisicao*` ao aprovar ou rejeitar. Se a Cloud Function de criação falhar após criar o lock mas antes de criar a requisição (ex.: timeout), o lock fica órfão; uma Cloud Function de limpeza agendada remove locks com mais de 24 horas **somente se não existir** uma requisição com status pendente vinculada àquele bem.

4.13 Impressão de Etiquetas de Frasco

Impressao_Etiqueta_Frasco

id

SERIAL **PK**

gerado_em

TIMESTAMP NOT NULL

gerado_por

INTEGER FK, NOT NULL

codigo_inicial

INTEGER NOT NULL

codigo_final

INTEGER NOT NULL

i Observação

Esta entidade registra cada sessão de impressão de etiquetas de frasco. *codigo_inicial* e *codigo_final* referem-se aos números sequenciais dos códigos de frasco no formato LCQUI-N (ex.: LCQUI-1 a LCQUI-50). Ver regra de geração de código em 7.2.9. A última impressão e o último código cadastrado são usados pelo modal de geração para oferecer ao gestor três opções: desde o último frasco cadastrado, desde a última impressão ou intervalo personalizado.

4.14 Contador**Contador_Codigo_Frasco***id*

VARCHAR(50) PK, Fixo como "singleton"

ultimo_codigo_gerado

INTEGER NOT NULL, Default 0

i Numeração Sem Gaps (Transação Atômica)

O código sequencial LCQUI-N dos frascos é gerado por uma transação nativa (*runTransaction*) no Firestore. Como o processo de cadastro físico de reagentes tem baixíssima frequência (muito inferior ao limite de 1 escrita/segundo), optou-se pela simplicidade e integridade matemática: a leitura do *ultimo_codigo_gerado*, o incremento e a criação do documento *Frasco_Reagente* ocorrem no mesmo bloco ACID. Isso elimina a complexidade acidental de bancos auxiliares (RTDB) e garante 100% de ausência de lacunas (*gaps*) na impressão de etiquetas (rollback automático em caso de erro na gravação do frasco).

4.15 Almoxarifado**Almoxarifado***id*

SERIAL PK

id_local

INTEGER FK, NOT NULL

nome

VARCHAR(100) NOT NULL

descricao

VARCHAR(500) NOT NULL

4.16 Substância Química

Substancia_Quimica

<i>id</i>	SERIAL	PK
<i>nome</i>	VARCHAR(150)	NOT NULL
<i>cas_number</i>	VARCHAR(15)	UNIQUE, NULL
<i>formula_quimica</i>	VARCHAR(50)	NULL
<i>ativo</i>	BOOLEAN	DEFAULT TRUE, NOT NULL

i Observação

A entidade *Substancia_Quimica* representa uma substância quimicamente identificável, independentemente de fabricante, concentração, pureza, lote ou frasco. O campo *cas_number* identifica a substância e permanece **UNIQUE** (permitindo múltiplos NULL, já que algumas substâncias podem não ter CAS cadastrado). Uma mesma substância pode participar da composição de diversas especificações de reagentes diferentes.

4.17 Resumo do Reagente / Solvente

Resumo_Reagente

<i>id</i>	SERIAL	PK
<i>nome</i>	VARCHAR(150)	NOT NULL
<i>tipo_substancia</i>	ENUM	PURA, MISTURA, NOT NULL
<i>natureza_quimica</i>	ENUM	ORGANICO, INORGANICO, ELEMENTO, HIBRIDO, NOT NULL
<i>requer_pesagem_frequente</i>	BOOLEAN	DEFAULT FALSE, NOT NULL
<i>qtd_em_que_e_considerado_escasso</i>	INTEGER	NOT NULL
<i>frequencia_pesagem_dias</i>	INTEGER	NULL

i Regra Fechada de Identidade do Resumo de Reagente

O *Resumo_Reagente* representa o item químico agrupador apresentado no catálogo do sistema, permitindo agrupar diferentes especificações comerciais ou formulações do mesmo reagente.

Dois reagentes pertencem ao **mesmo** *Resumo_Reagente* quando compartilham a mesma identidade química nominal de catálogo (ex.: “Ácido Clorídrico”, “Etanol”, “Hidróxido de Sódio”).

- Variações de pureza (P.A., Técnico, Grau HPLC) $\Rightarrow$ **mesmo resumo**, especificações distintas.
- Variações de concentração (HCl 37%, HCl 10%, Etanol 99.5%, Etanol 70%) $\Rightarrow$ **mesmo resumo**, especificações distintas.
- Fabricante, código de catálogo e densidade $\Rightarrow$ atributos de especificação, **mesmo resumo**.

Devem ser cadastrados como **resumos diferentes**:

- Espécies químicas distintas com fórmulas ou números CAS bases diferentes (ex.: Sulfato de Cobre II vs Sulfato de Ferro II);
- Soluções formuladas consagradas com finalidade analítica própria (ex.: “Reagente de Benedict”, “Solução de Fehling A”, “Solução de Lugol”) $\Rightarrow$ cada uma constitui um *Resumo_Reagente* próprio do tipo MISTURA.

Os campos *medida_total_usado* e *unidade_de_medida* foram removidos desta entidade: ambos são métricas agregadas derivadas de *Frasco_Reagente* e *Empres-timo_Reagente*, e devem existir apenas na view materializada descrita na seção *Métricas Agregadas*, nunca como coluna editável de uma tabela cadastral.

A unidade de medida (ml ou g) passa a ter fonte única em *Especificacao_Reagente*, evitando duplicidade e divergência entre os níveis do modelo.

A densidade, o estado físico e a unidade de medida são propriedades da *Especificacao_Reagente*. Assim, HCl 37% e HCl 10% podem compartilhar o mesmo *Resumo_Reagente*, mas devem possuir especificações distintas. A regra de identidade do resumo é determinada pelo item químico que o catálogo pretende agrupar, e não por uma cópia de propriedades operacionais da especificação.

natureza_quimica classifica o item do catálogo como um todo (não uma *Substancia_Quimica* individual): **ELEMENTO** para substâncias simples (não se classificam como orgânicas nem inorgânicas), **ORGANICO/INORGANICO** para os demais casos, e **HIBRIDO** para fluidos complexos que não se decompõem em uma lista simples de

substâncias em *Composicao_Reagente* (ex.: Soro Fetal Bovino). Como HÍBRIDO não é derivável a partir das substâncias componentes, o campo é curado no nível do resumo, e não em *Substancia_Quimica*, evitando duas fontes de verdade diferentes para itens PUROS e para MISTURA/HÍBRIDO. O rótulo exibido na interface (ex.: “Híbrido”, “Complexo” ou “Biológico”) é independente do valor salvo no banco e pode ser alterado livremente.

O campo *letra_inicial*, presente em versões anteriores deste documento, foi removido do modelo 3FN: por ser derivável do próprio *nome*, sua persistência aqui violaria a forma normal. Ele reaparece como uma denormalização proposital exclusiva da implementação em Firestore – ver seção *Notas de Mapeamento para Firestore*.

4.18 Composição do Reagente

Composicao_Reagente

<i>id</i>	SERIAL	PK
<i>id_especificacao_reagente</i>	INTEGER	FK, NOT NULL
<i>id_substancia_quimica</i>	INTEGER	FK, NOT NULL
<i>valor_composicao</i>	NUMERIC(10,5)	NULL
<i>tipo_concentracao</i>	ENUM	M_M, V_V, M_V, MOL_L, MOL_KG, PPM, PPB
<i>unidade</i>	VARCHAR(20)	NULL

i Observação

A entidade *Composicao_Reagente* representa a relação entre uma especificação de reagente cadastrada e as substâncias químicas que a compõem.

A relação é do tipo N:N:

$$Especificacao_Reagente \leftrightarrow Substancia_Quimica$$

A composição foi movida do nível de *Resumo_Reagente* para o nível de *Especificacao_Reagente*, pois a concentração (37%, 10%, 1 mol/L etc.) é uma característica que varia entre especificações de um mesmo resumo, não uma propriedade do resumo em si. Vincular a composição ao resumo impediria registrar corretamente, por exemplo, HCl 37% e HCl 10% sob o mesmo *Resumo_Reagente* com valores de composição diferentes.

Uma substância pura normalmente terá apenas uma linha associada, enquanto uma mistura poderá possuir diversas substâncias.

O campo *unidade* é armazenado como texto livre para suportar tanto um valor único (ex.: “37”) quanto um intervalo no formato “MIN-MAX” (ex.: “15-20”), usado quando a especificação do fabricante declara uma faixa de concentração em vez de um valor fixo. A validação desse formato (numérico simples ou X-Y com X < Y) é responsabilidade da camada de aplicação (*frontend*); o banco não impõe *constraint* de formato sobre este campo.

Regra de negócio

UNIQUE(id_especificacao_reagente, id_substancia_quimica) – impede duas linhas de composição para a mesma substância dentro da mesma especificação.

4.19 Especificação do Reagente / Solvente

Especificacao_Reagente

<i>id</i>	SERIAL	PK
<i>id_resumo_reagente</i>	INTEGER	FK, NOT NULL
<i>descricao</i>	VARCHAR(150)	NOT NULL
<i>fabricante</i>	VARCHAR(150)	NULL
<i>codigo_produto_fabricante</i>	VARCHAR(100)	NULL
<i>grau_pureza</i>	VARCHAR(30)	NULL
<i>densidade</i>	NUMERIC(8,4)	NULL
<i>estado_fisico</i>	ENUM	SOLIDO, LIQUIDO, NOT NULL
<i>unidade_de_medida</i>	ENUM	ml, g, NOT NULL
<i>classe_inflamabilidade</i>		ENUM
	NAO_INFLAMAVEL, CLASSE_1, CLASSE_2, CLASSE_3, NOT NULL	
<i>eh_controlado_pf</i>	BOOLEAN	DEFAULT FALSE, NOT NULL
<i>eh_controlado_eb</i>	BOOLEAN	DEFAULT FALSE, NOT NULL
<i>link_fds_fispq</i>	VARCHAR(255)	NULL

i Observação

Especificacao_Reagente passa a ser a única fonte de verdade para *unidade_de_medida*. Os demais níveis (*Frasco_Reagente*, *Emprestimo_Reagente*) referenciam essa unidade por meio da cadeia de FKs, em vez de armazenar uma cópia própria.

GASOSO foi removido de *estado_fisico* nesta versão: nenhum reagente gasoso está listado para o almoxarifado atual, e reagentes gasosos precisam de campos e método de medição próprios (pressão, não peso em balança) em vez de reaproveitar os campos já pensados para sólido/líquido. O estudo detalhado para reintrodução está na seção *Implementações em Estudo para Versões Futuras*.

4.20 Lote

Lote

<i>id</i>	SERIAL	PK
<i>id_especificacao_reagente</i>	INTEGER	FK, NOT NULL
<i>data_aquisicao</i>	TIMESTAMP	NOT NULL

<i>qtd_frascos_comprados</i>	INTEGER	DEFAULT 0, NOT NULL
<i>nome_fornecedor</i>	VARCHAR(80)	NOT NULL
<i>numero_lote</i>	VARCHAR(100)	NOT NULL
<i>nota_fiscal</i>	VARCHAR(100)	NOT NULL
<i>data_fabricacao</i>	DATE	NOT NULL
<i>data_validade</i>	DATE	NOT NULL

Regra de negócio

UNIQUE(id_especificacao_reagente, nome_fornecedor, numero_lote).

i Observação

qtd_frascos_comprados é dado de entrada (o que consta na nota fiscal) e permanece nesta tabela. Já *qtd_frascos_cadastrados* – quantos frascos deste lote já foram individualmente registrados em *Frasco_Reagente* – é uma contagem agregada (`COUNT(*) FROM Frasco_Reagente WHERE id_lote = X`) e, por isso, **não** é coluna desta tabela cadastral: ela vive exclusivamente em *Lote_Materializado* (ver seção *Materialized (Views)*), mantida por mecanismo transacional a cada frasco cadastrado ou removido.

4.21 Frasco de Reagente / Solvente

Frasco_Reagente

<i>id</i>	SERIAL	PK
<i>id_almoxxarifado</i>	INTEGER	FK, NOT NULL
<i>id_lote</i>	INTEGER	FK, NULL
<i>id_especificacao_reagente</i>	INTEGER	FK, NULL
<i>detalhe_local_armazenamento</i>	TEXT	NULL
<i>codigo_frasco</i>	TEXT	UNIQUE, NOT NULL
<i>data_ultima_pesagem</i>	TIMESTAMP	NULL
<i>conteudo_nominal</i>	NUMERIC(10,3)	NOT NULL
<i>peso_no_cadastrado</i>	NUMERIC(10,3)	NOT NULL
<i>peso_atual</i>	NUMERIC(10,3)	NOT NULL
<i>peso_frasco_vazio</i>	NUMERIC(10,3)	NULL
<i>medida_usada</i>	NUMERIC(10,3)	DEFAULT 0, NOT NULL

<i>data_abertura</i>	DATE	NULL
<i>validade_fechado</i>	DATE	NULL
<i>validade_efetiva</i>	DATE	NULL
<i>validade_desconhecida</i>	BOOLEAN	DEFAULT FALSE, NOT NULL
<i>validade_apos_aberto_dias</i>	INTEGER	NULL
<i>estado_fisico_frasco</i>	ENUM	FECHADO, ABERTO, VAZIO, QUEBRADO, DESCARTADO, NOT NULL
<i>disponibilidade</i>	ENUM	DISPONIVEL, EMPRESTADO, NOT NULL
<i>vencido</i>	BOOLEAN	DEFAULT FALSE, NOT NULL
<i>em_quarentena</i>	BOOLEAN	DEFAULT FALSE, NOT NULL
<i>uso_vencido_autorizado</i>	BOOLEAN	DEFAULT FALSE, NOT NULL
<i>detalhe_status</i>	TEXT	NULL
<i>cadastrado_em</i>	TIMESTAMP	NOT NULL
<i>cadastrado_por</i>	INTEGER	FK, NOT NULL

Constraint de Identidade Química do Frasco

CHECK (id_lote IS NOT NULL OR id_especificacao_reagente IS NOT NULL) – pelo menos um dos dois deve existir. Quando *id_lote* é informado, *id_especificacao_reagente* fica NULL (a especificação é derivável via *Lote.id_especificacao_reagente*). Quando *id_lote* é NULL, *id_especificacao_reagente* é obrigatório para manter o caminho até a densidade, unidade de medida e resumo do reagente.

i Observação

conteudo_nominal (antigo *capacidade_nominal*) representa o conteúdo declarado no cadastro: para líquidos, é o volume nominal em mL; para sólidos, é a massa nominal em g. A renomeação elimina a ambiguidade semântica entre volume e massa.

peso_no_cadastrado é o peso total (frasco + conteúdo) registrado no momento do cadastro. Este campo é **imutável** após o cadastro – nunca é sobrescrito. *peso_atual* é o peso total corrente, inicializado com o valor de *peso_no_cadastrado* e atualizado a cada devolução de empréstimo ou pesagem de rotina.

Os campos *peso_no_cadastrado*, *peso_atual*, *peso_frasco_vazio* e *medida_usada* são sempre leituras de balança em gramas, independentemente de o conteúdo ser sólido ou líquido, já que a operação real do laboratório consiste em pesar o frasco;

o volume, quando aplicável, é sempre derivado por conversão via densidade.

O campo *unidade_de_medida*, antes duplicado nesta entidade, foi removido: a unidade do produto (ml ou g) é obtida via *Frasco_Reagente* → *Lote* → *Especificacao_Reagente* (ou diretamente via *id_especificacao_reagente* quando o lote é NULL).

O antigo campo único *status* (com valores combinados como VENCIDO_DISPONIVEL, VENCIDO_EMPRESTADO) foi decomposto em dimensões ortogonais – *estado_fisico_frasco*, *disponibilidade*, *vencido* e *em_quarentena* – evitando o crescimento combinatorial do enum e simplificando consultas e regras de notificação.

O campo *id_usuario_em_uso* foi removido: quem está com o frasco no momento é obtido consultando *Emprestimo_Reagente* pelo empréstimo ativo (*status* EM_USO ou ATRASADO) daquele *id_frasco_reagente*.

validade_fechado é mantido de forma independente de *Lote.data_validade* porque *id_lote* é opcional: um frasco pode ser cadastrado sem lote associado, e mesmo quando o lote existe, o formulário apenas pré-preenche este campo a partir dele – o valor permanece editável e não é uma cópia somente-leitura.

O campo *uso_vencido_autorizado* fecha a distinção de negócio entre um frasco vencido que permanece apto ao uso didático e um frasco vencido que aguarda descarte.

4.22 Histórico do Frasco de Reagente / Solvente

Historico_Frasco_Reagente

id

SERIAL PK

id_frasco_reagente

INTEGER FK, NOT NULL

id_almoxxarifado

INTEGER FK, NOT NULL

id_gestor

INTEGER FK, NOT NULL

Para ações automáticas do servidor (jobs de vencimento, atraso), usar o UID sentinela ____SISTEMA____.

tipo

ENUM

CADASTRO, SAIU, ENTROU, FICOU_VAZIO, QUEBROU, FOI_DESCARTADO, VENCEU, INVENTARIO_ROTINA, EVAPORACAO, AJUSTE, CONCLUSAO, ENTROU_EM_QUARENTENA, LIBERADO_QUARENTENA, PENDENTE_DE_DESCARTE, USO_VENCIDO_AUTORIZADO

NOT NULL ***campo_ajustado***

VARCHAR(30) NULL

id_emprestimo_reagente

INTEGER FK, NULL

peso_anterior

NUMERIC(10,3) NULL

peso_novo

NUMERIC(10,3) NULL

medida_ajustada

NUMERIC(10,3) NULL

unidade_medida_ajustada

ENUM ml,g, NOT NULL

timestamp

TIMESTAMP NOT NULL

i Observação

peso_anterior e *peso_novo* são sempre leituras de balança em gramas. Já *unidade_medida_ajustada* qualifica exclusivamente *medida_ajustada*, indicando se o ajuste concluído é reportado em massa (g, sólidos) ou volume (ml, líquidos convertidos por densidade).

4.23 Empréstimo de Reagente / Solvente**Emprestimo_Reagente***id*

SERIAL PK

id_frasco_reagente

INTEGER FK, NOT NULL

id_usuario_retirou

INTEGER FK, NOT NULL

status

ENUM EM_USO, ATRASADO, DEVOLVIDO, DEVOLVIDO_COM_ATRASO, NOT NULL

data_retirada

TIMESTAMP NOT NULL

data_devolucao_prevista

DATE NOT NULL

data_devolucao_efetuada

TIMESTAMP NULL

id_local_usado

INTEGER FK, NOT NULL

id_usuario_devolveu

INTEGER FK, NULL

peso_saida

NUMERIC(10,3) NOT NULL

peso_retorno

NUMERIC(10,3) NULL

medida_utilizada

NUMERIC(10,3) NULL

unidade_medida_utilizada

ENUM ml,g, NOT NULL

peso_perda_evaporacao

NUMERIC(10,3) DEFAULT 0, NOT NULL

uso_vencido_aceito

BOOLEAN DEFAULT FALSE, NOT NULL

finalidade_uso

ENUM PESQUISA, DIDATICO_DEMONSTRACAO, OUTRO

NOT NULL

i Observação

peso_saida e *peso_retorno* são sempre leituras de balança em gramas. *unidade_medida_utilizada* qualifica exclusivamente *medida_utilizada* – o consumo calculado, reportado em massa (g) para sólidos ou em volume (ml) para líquidos, via $V = \Delta m / \rho$.

4.24 Métricas Agregadas do Reagente**i Observação**

As seguintes informações devem ser tratadas preferencialmente como **views materializadas**, **consultas agregadas** ou **campos derivados mantidos por mecanismo transacional**:

- *medida_total_usado* e *unidade_de_medida* (agregados por *Resumo_Reagente*);
- *qtd_frascos_fechados_disponiveis*;
- *qtd_frascos_abertos_disponiveis*;
- *qtd_frascos_em_uso*;
- *qtd_frascos_vazios*;
- *qtd_frascos_descartados*;
- *qtd_frascos_vencidos*;
- *qtd_frascos_quebrados*.

4.25 Turma**Turma**

<i>id</i>	SERIAL	PK
<i>id_materia</i>	INTEGER	FK, NOT NULL
<i>id_professor</i>	INTEGER	FK, NOT NULL
<i>status</i>	ENUM	Ativo, Arquivada, NOT NULL
<i>nome_turma</i>	VARCHAR(100)	NOT NULL
<i>ano</i>	INTEGER	NOT NULL
<i>semestre</i>	SMALLINT	CHECK (semestre IN (1,2)), NOT NULL

capacidade

INTEGER NOT NULL

codigo_turma

VARCHAR(20) UNIQUE, NOT NULL

data_criacao

TIMESTAMP NOT NULL

i Observação

Os campos *ano* e *semestre* representam o período letivo da turma (ex.: 2026/1). Ambos são exibidos na listagem de turmas arquivadas do professor – eliminando a necessidade de parsing do *nome_turma* ou inferência a partir de *data_criacao*. O modal de criação de turma solicita esses dois campos além dos demais.

RN-TUR-01: Controle de Capacidade da Turma

O ingresso de aluno por código de turma exige obrigatoriamente que a quantidade de alunos matriculados ativos seja estritamente menor que a capacidade da turma:

$$\text{COUNT}(\text{Aluno_x_Turma WHERE id_turma} = X) < \text{Turma.capacidade}$$

Caso a capacidade seja atingida, o sistema rejeita o ingresso por código. Apenas o professor responsável pode ultrapassar a capacidade mediante envio de convite nominal por e-mail, registrando justificativa em auditoria.

4.26 Historico de Alunos na Turma**Historico_Alunos_Turma***id*

SERIAL PK

id_turma

INTEGER FK, NOT NULL

tipo

ENUM inclusao_aluno, exclusao_aluno, NOT NULL

id_aluno

INTEGER FK, NOT NULL

timestamp

TIMESTAMP NOT NULL

4.27 Historico de Posts na Turma

Historico_Posts_Turma

<i>id</i>	SERIAL	PK
<i>id_post</i>	INTEGER	FK, NOT NULL
<i>novo_titulo</i>	VARCHAR(150)	NULL
<i>novo_id_roteiro</i>	INTEGER	FK, NULL
<i>nova_descricao</i>	TEXT	NULL
<i>antigo_titulo</i>	VARCHAR(150)	NULL
<i>antigo_id_roteiro</i>	INTEGER	FK, NULL
<i>antiga_descricao</i>	TEXT	NULL
<i>timestamp</i>	TIMESTAMP	NOT NULL

4.28 Historico de Comentario em um Post

Historico_Comentario

<i>id</i>	SERIAL	PK
<i>id_comentario</i>	INTEGER	FK, NOT NULL
<i>novo_texto</i>	TEXT	NOT NULL
<i>texto_antigo</i>	TEXT	NOT NULL
<i>editado_em</i>	TIMESTAMP	NOT NULL
<i>editado_por</i>	INTEGER	FK, NOT NULL

4.29 Roteiro de Experimento

Roteiro_Experimento

<i>id</i>	SERIAL	PK
<i>id_professor_upload</i>	INTEGER	FK, NOT NULL
<i>file_url</i>	TEXT	NOT NULL
<i>nome</i>	VARCHAR(150)	NOT NULL
<i>descricao</i>	TEXT	NOT NULL

4.30 Roteiro Professor Compartilhado

Roteiro_Professor_Compartilhado

<i>id_roteiro_experimento</i>	INTEGER	PK, FK
<i>id_professor</i>	INTEGER	PK, FK

4.31 Professor

Professor

<i>id_usuario</i>	INTEGER	PK, FK
<i>centro</i>	ENUM	CCT, CCTA, CBB, CCH, NOT NULL
<i>laboratorio</i>	VARCHAR(100)	NOT NULL

4.32 Materia

Materia

<i>id</i>	SERIAL	PK
<i>nome</i>	VARCHAR(100)	NOT NULL
<i>codigo_materia</i>	VARCHAR(10)	UNIQUE, NOT NULL

4.33 Tabela Professor × Materia (N para N)

Professor_x_Materia

<i>id_professor</i>	INTEGER	PK, FK
<i>id_materia</i>	INTEGER	PK, FK

Regra de negócio

A chave primária composta (*id_professor*, *id_materia*) impede vincular o mesmo professor duas vezes à mesma matéria.

4.34 Aluno

Aluno

<i>id_usuario</i>	INTEGER	PK, FK
<i>numero_matricula</i>	VARCHAR(20)	UNIQUE, NOT NULL

4.35 Bolsista

Bolsista

<i>id_usuario</i>	INTEGER	PK, FK
-------------------	---------	--------

4.36 Convite para Aluno

Convite_Aluno

<i>id</i>	SERIAL	PK
<i>id_turma</i>	INTEGER	FK, NULL
<i>email</i>	VARCHAR(150)	NOT NULL
<i>convidado_em</i>	TIMESTAMP	NOT NULL
<i>status</i>	ENUM	pendente, aceitado, expirado, NOT NULL
<i>expira_em</i>	TIMESTAMP	NOT NULL
<i>convidado_por</i>	INTEGER	FK, NOT NULL
<i>numero_matricula</i>	VARCHAR(20)	NULL

Regra de negócio

A combinação *email* + *id_turma* não pode possuir mais de um convite *pendente*. Um mesmo e-mail pode possuir convites para turmas diferentes e um aluno pode participar de várias turmas; por isso, o e-mail não é globalmente único nesta entidade.

i Observação

numero_matricula aqui é opcional e **não é UNIQUE**, pois serve apenas para pré-declarar a matrícula quando o professor a conhece no momento do convite. Ao aceitar o convite, o sistema cria as linhas *Usuario* e *Aluno* correspondentes,

copiando *email* e *numero_matricula* para a nova linha de *Aluno*, onde a restrição **UNIQUE** é rigorosamente aplicada.

4.37 Notificação (Unificada)

Notificacao

<i>id</i>	SERIAL	PK
<i>id_destinatario</i>	INTEGER	FK, NOT NULL
<i>papel_destinatario</i>	ENUM	Aluno, Professor, Gestor_Bens_Patrimoniais, Gestor_Almozarifado
NOT NULL <i>tipo</i>	ENUM	ADICIONADO, POST, COMENTARIO, REMOVIDO, TURMA_ARQUIVADA, TURMA_DESARQUIVADA, REQUISICAO_BEM, DATA_DEVOLUCAO_REAGENTE, ROTEIRO_COMPARTILHADO, REQUISICAO_EDICAO_BEM, REQUISICAO_ADICAO_BEM, BEM_INSERTIVEL, ENTREGA_ATRASADA, FRASCOS_VAZIOS, FRASCOS_QUEBRADOS, FRASCOS_VENCIDOS, FRASCOS_A_SEREM_PESADOS, FRASCOS_EM_QUARENTENA, REAGENTE_ESCASSO
NOT NULL <i>id_quem_fez_acao</i>	INTEGER	FK, NULL
<i>id_turma</i>	INTEGER	FK, NULL
<i>quantidade</i>	INTEGER	NULL
<i>entidade_alvo</i>	VARCHAR(40)	NOT NULL
<i>id_alvo</i>	VARCHAR(200)	NOT NULL
<i>lida</i>	BOOLEAN	DEFAULT FALSE, NOT NULL
<i>lida_em</i>	TIMESTAMP	NULL
<i>emitida_em</i>	TIMESTAMP	NOT NULL
<i>expira_em</i>	TIMESTAMP	NOT NULL

i Unificação das Notificações

As quatro tabelas de notificação anteriores (*Notificacao_para_Aluno*, *Notificacao_para_Professor*, *Notificacao_para_Gestor_de_bens_Patrimoniais*, *Notificacao_para_Gestor_de_Reagentes*) foram consolidadas em uma única entidade *Notificacao*. O campo *papel_destinatario* indica em qual contexto de papel a notificação deve ser exibida.

No Firestore, esta entidade é implementada como sub-coleção *Usuarios/{uid}/Notificacoes*, eliminando a necessidade de queries em múltiplas coleções para usuários multi-role. A ação de “Limpar tudo” na interface

marca lida = true e lida_em = NOW() em todas as notificações ativas do usuário, preservando o histórico de auditoria. Os campos *entidade_alvo* e *id_alvo* viabilizam a navegação direta (*deep linking*) para o objeto de origem ao clicar no aviso.

4.38 Tabela Aluno Turma (N para N)

Aluno_x_Turma

id_aluno

INTEGER PK, FK

id_turma

INTEGER PK, FK

Regra de negócio

A chave primária composta (*id_aluno*, *id_turma*) impede matricular o mesmo aluno duas vezes na mesma turma.

4.39 Local

Local

id

SERIAL PK

predio

VARCHAR(30) NOT NULL

andar

VARCHAR(10) NOT NULL

sala

VARCHAR(30) NOT NULL

criado_por

INTEGER FK, NOT NULL

criado_em

TIMESTAMP NOT NULL

Regra de negócio

A combinação de prédio, andar e sala deve ser única (*UNIQUE(predio, andar, sala)*).

4.40 Post

Post

<i>id</i>	SERIAL	PK
<i>id_professor</i>	INTEGER	FK, NOT NULL
<i>id_turma</i>	INTEGER	FK, NOT NULL
<i>id_roteiro_experimento</i>	INTEGER	FK, NULL
<i>titulo</i>	VARCHAR(150)	NOT NULL
<i>descricao</i>	TEXT	NOT NULL
<i>criado_em</i>	TIMESTAMP	NOT NULL

4.41 Comentario

Comentario

<i>id</i>	SERIAL	PK
<i>id_post</i>	INTEGER	FK, NOT NULL
<i>id_usuario</i>	INTEGER	FK, NOT NULL
<i>texto</i>	TEXT	NOT NULL
<i>criado_em</i>	TIMESTAMP	NOT NULL

4.42 Registro de Auditoria

Registro_de_Auditoria

<i>id</i>	SERIAL	PK
<i>id_usuario</i>	INTEGER	FK, NOT NULL
<i>acao</i>	VARCHAR(200)	NOT NULL
<i>tipo_entidade_sofre_acao</i>	ENUM	
TURMA, ALUNO, PROFESSOR, ALMOXARIFADO, RESUMO_REAGENTE, ESPECIFICACAO_REAGENTE, LOTE, FRASCO_REAGENTE, EMPRESTIMO_REAGENTE, BEM_PATRIMONIAL, LOCAL, MATERIA, COMENTARIO, POST, ROTEIRO, REQUISICAO_ADICAO_BEM, REQUISICAO_EDICAO_BEM, USUARIO		
NOT NULL <i>id_do_objeto_da_entidade</i>	VARCHAR(200)	NOT NULL
<i>acao_feita_em</i>	TIMESTAMP	NOT NULL
<i>metadata</i>	JSONB	NOT NULL

i Observação

acao e *id_do_objeto_da_entidade* tinham o tamanho 200 escrito como se fosse um selo de restrição em vez de parte do tipo; corrigido para `VARCHAR(200)`. O valor `REAGENTE` do enum era ambíguo diante da hierarquia real (*Resumo_Reagente*, *Especificacao_Reagente*, *Lote*, *Frasco_Reagente* já existiam como tabelas distintas, e *FRASCO_REAGENTE* já aparecia separadamente) – substituído pelos valores granulares correspondentes, e *EMPRESTIMO_REAGENTE*, *ALMOXARIFADO* e *USUARIO* foram adicionados por também serem alvo de auditoria (troca de papel, criação de almoxarifado, edição de perfil).

4.43 Especificação de Integridade do Modelo 3FN

Esta subseção fecha as propriedades lógicas que devem permanecer verdadeiras antes de qualquer denormalização específica do Firestore.

4.43.1 Cardinalidades

Relacionamento	Card.	Regra
Usuario – cada papel específico	0..1 por papel	Um usuário pode ter no máximo uma linha em cada tabela de papel; várias tabelas de papel podem estar associadas ao mesmo usuário conforme as combinações permitidas.
Gestor Almoхарifado – Almoхарifado	N:N	A mesma associação não pode ocorrer duas vezes.
Resumo Bem – Bem Patrimonial	1:N	Cada bem pertence a um resumo.
Local – Bem Patrimonial	1:N	Cada bem possui um local cadastrado.
Resumo Reagente – Especificação	1:N	Um resumo agrupa especificações; cada especificação pertence a um resumo.
Especificação – Substância	N:N	A composição liga múltiplas substâncias a múltiplas especificações.
Especificação – Lote	1:N	Um lote pertence a uma única especificação.
Lote – Frasco	1:N opcional	Um frasco pode ter lote NULL quando o lote não for conhecido.
Almoхарifado – Frasco	1:N	Cada frasco está em exatamente um almoхарifado.
Frasco – Empréstimo	1:N	Histórico completo; no máximo um empréstimo ativo por frasco.
Turma – Aluno	N:N	Alunos podem participar de várias turmas (via sub-coleção
textttTurma/{id}/Alunos).		
Turma – Post	1:N	Cada post pertence a uma turma e ao professor autor.
Post – Comentário	1:N	Cada comentário

4.43.2 Nullabilidade

Todo atributo é **NOT NULL** por padrão. Um campo só pode ser NULL quando isso representa uma ausência legítima do domínio. As exceções são:

- foto de perfil;
- documento de baixa antes da baixa patrimonial;
- campos de resposta de requisição antes da resposta;
- campos opcionais de especificação (fabricante, código do fabricante, FDS/FISPQ e demais dados não disponíveis);
- CAS e fórmula química quando não cadastrados;
- frequência de pesagem quando o reagente não requer pesagem frequente;
- lote e dados de validade/pesagem quando desconhecidos ou ainda não aplicáveis;
- validade_efetiva antes da abertura quando sua definição depende do prazo após abertura;
- dados de devolução de empréstimo antes da devolução;
- id_local da linha agregada “Geral” no resumo patrimonial diário;
- id_turma da notificação de professor quando o tipo não é COMENTARIO;
- atributos de composição cuja ausência é necessária para representar faixa ou notação textual do fabricante.

4.43.3 Chaves e constraints

- Usuario.email, Bem_Patrimonial.numero_patrimonio, Frasco_Reagente.codigo_frasco, Turma.codigo_turma, Materia.codigo_materia e CAS não nulo são únicos.
- As tabelas associativas usam chave primária composta por suas duas FKs.
- Composicao_Reagente usa UNIQUE(id_especificacao_reagente, id_substancia_quimica).
- Lote usa UNIQUE(id_especificacao_reagente, nome_fornecedor, numero_lote), evitando a identificação de dois lotes equivalentes do mesmo fornecedor e produto.

- `Convite_Aluno` permite o mesmo e-mail em turmas diferentes, mas impede mais de um convite PENDENTE para a mesma combinação de e-mail normalizado e turma; para convite global (*id_turma* NULL), impede mais de um convite PENDENTE para o mesmo e-mail normalizado. No PostgreSQL, isso é representado por dois índices únicos parciais: `UNIQUE(email_normalizado, id_turma) WHERE id_turma IS NOT NULL AND status = 'pendente'` e `UNIQUE(email_normalizado) WHERE id_turma IS NULL AND status = 'pendente'`. No Firestore, a Cloud Function deve realizar a mesma validação dentro de transação.
- `Requisicao_Edicao_Bem_Patrimonial` permite no máximo uma requisição PENDENTE por bem.
- Todos os valores quantitativos de massa, volume, capacidade, contagens, mês, dia de frequência e quantidades agregadas devem obedecer aos limites não negativos ou positivos definidos pelas regras do domínio.
- Datas que representam intervalos devem respeitar sua ordem temporal; exemplos: fabricação $\leq$ validade e devolução $\geq$ retirada.

4.43.4 Regras químicas fechadas

- **PURA** e **MISTURA** são tipos de substância do resumo. **MISTURA** exige composição na especificação; **PURA** não depende de múltiplas linhas de composição.
- A concentração, quando aplicável, é atributo da composição da especificação, e não do resumo.
- A densidade é atributo da especificação. É obrigatória e positiva para **LIQUIDO** e pode ser NULL para **SOLIDO**.
- **LIQUIDO** usa ml como unidade operacional; **SOLIDO** usa g. O par estado/unidade deve ser sempre coerente.
- Duas especificações com o mesmo nome podem coexistir sob o mesmo resumo quando diferirem em concentração, densidade, fabricante ou código do produto e representarem o mesmo item agrupador. Diferença de especificação não implica automaticamente novo resumo.
- Gases estão fora do escopo da V1. Não se deve misturar sua futura regra de pressão com os campos de pesagem de sólidos e líquidos.

4.43.5 Máquina de estados e invariantes do frasco

Estado/ação	Invariante
FECHADO + DISPONIVEL	Pode ser retirado ou aberto.
ABERTO + DISPONIVEL	Pode ser retirado ou ajustado por pesagem.
EMPRESTADO	Deve possuir exatamente um empréstimo ativo EM_USO ou ATRASADO.
VAZIO/QUEBRADO	Não pode estar EMPRESTADO e deve aparecer como pendente de descarte.
QUARENTENA	Exige detalhe de motivo e bloqueia nova retirada.
VENCIDO + uso_vencido_autorizado = false	Exibe PENDENTE DE DESCARTE quando não estiver em quarentena; bloqueia nova retirada.
VENCIDO + uso_vencido_autorizado = true	Pode permanecer DISPONIVEL para uso excepcional após confirmação explícita no empréstimo.
DESCARTADO	É terminal; não retorna a DISPONIVEL.
Devolução	Só pode ocorrer para empréstimo EM_USO ou ATRASADO e o peso de retorno não pode exceder o peso de saída.

4.43.6 Fechamento dos estados automáticos

vencido é definido pelo servidor a partir da validade efetiva. Na abertura do frasco, *validade_efetiva* é recalculada conforme a regra de validade pós-abertura. *ATRASADO* é promovido pelo job servidor quando a data de devolução prevista é ultrapassada. Nenhum desses estados pode ser inventado pelo frontend. O vencimento também é recalculado sincronicamente em operações críticas de cadastro e devolução, para que o resultado não dependa de o job diário já ter executado.

4.44 Checklist de Fechamento do Domínio e do Modelo

Os onze pontos que precisavam ser resolvidos antes da modelagem física foram consolidados da seguinte forma:

- 1. Matriz completa de permissões:** formalizada na seção de Stakeholders e referenciada como fonte normativa de autorização.
- 2. Chefe Geral e multi-role:** Chefe Geral é exclusivo por conta; eventual dupla função institucional exige contas independentes.

3. **Obrigatoriedade de campos:** formalizada por contexto, estado, tipo de substância e operação.
4. **Densidade:** reposicionada semanticamente como propriedade de Especificacao_Reagente e usada nos cálculos sem duplicação no resumo.
5. **Máquina de estados:** estados do frasco e transições de retirada, devolução, quarentena, vencimento, esvaziamento, quebra e descarte foram explicitados.
6. **NULL/NOT NULL:** todo campo passa a ter semântica explícita, com NULL restrito a ausências legítimas do domínio.
7. **Cardinalidades:** relacionamentos 1:N, N:N e especializações de papel foram consolidados.
8. **Constraints:** unicidades, chaves compostas, regras temporais e limites quantitativos foram formalizados, indicando quais precisarão ser simulados por transação no Firestore.
9. **Métricas e unidades:** limiar de escassez passa a significar quantidade de frascos; massa de balança e consumo convertido permanecem conceitos distintos.
10. **Gases:** explicitamente fora do escopo da V1, com a extensão futura isolada para não contaminar o modelo atual.
11. **Vencimento e destino:** cadastro, abertura e devolução distinguem explicitamente quarentena, pendência de descarte e uso vencido autorizado.

i Critério de passagem para a modelagem Firestore

O modelo 3FN é considerado fechado quando cada requisito funcional consegue ser rastreado até entidade, atributo, constraint e fluxo correspondente sem depender de uma suposição não documentada. A seção seguinte pode então introduzir apenas as denormalizações necessárias às consultas do Firestore, sem alterar a semântica do modelo relacional.

5 Notas de Mapeamento para Firestore

i Por que esta seção existe

A modelagem da seção anterior é deliberadamente relacional e normalizada (3FN), pensada para ser lida e avaliada como um modelo de dados independente de tecnologia. O banco real do LCQUI, porém, é o **Firestore** (Firebase), um banco de documentos sem JOIN e sem LIKE/busca por substring. Esta seção documenta

as denormalizações propositas necessárias para a implementação – nenhuma delas deve ser lida como parte do modelo 3FN, e nenhuma delas deve alterar a seção 4.

5.1 Reintrodução de *letra_inicial*

O campo *letra_inicial*, removido de *Resumo_Reagente* por ser derivável de *nome* (violaria 3FN), volta a existir como campo próprio **apenas** no documento Firestore da coleção *Resumo_Reagente*. Motivo: o Firestore não tem LIKE/busca por substring, então a busca alfabética do catálogo precisa de um campo indexável para filtro de igualdade (ou de *range query* por prefixo). Como esse filtro normalmente é combinado com outros filtros de igualdade na mesma consulta (estado físico, natureza química), um campo de igualdade simples é mais previsível dentro das limitações de índice composto do Firestore do que uma *range query* de prefixo sobre *nome* combinada com filtros adicionais.

Consistência obrigatória

letra_inicial deve ser recalculado e reescrito pela mesma *Cloud Function* que grava *nome*, nunca aceito diretamente do cliente – ver princípio “nunca confiar no *client-side*” na seção *Tecnologia e Relatórios*.

5.2 Espelhamento de *estado_fisico*

estado_fisico tem fonte única em *Especificacao_Reagente* no modelo 3FN. Se a tela de busca do catálogo (*Resumo_Reagente*) precisar filtrar por estado físico junto com *natureza_quimica* e *letra_inicial* na mesma consulta, *estado_fisico* também precisa ser espelhado como campo próprio no documento Firestore de *Resumo_Reagente* – mesma lógica de *letra_inicial*.

5.3 Espelhamento de *nome_equipamento* e Localização em *Bem_Patrimonial*

No modelo 3FN, o nome do tipo de bem pertence exclusivamente a *Resumo_Bem_Patrimonial.nome*; *Bem_Patrimonial* mantém apenas *id_resumo_bem_patrimonial* e *id_local*. No Firestore, porém, consultas de listagem e relatórios frequentemente partem diretamente de *Bem_Patrimonial*, sem um JOIN para resolver o resumo ou o prédio. Por isso, o documento Firestore de *Bem_Patrimonial* deve conter os campos denormalizados *nome_equipamento*, *predio*, *andar* e *sala*.

Fonte única de verdade para o nome do patrimônio

nome_equipamento é exclusivamente um campo de leitura derivado de *Resumo_Bem_Patrimonial.nome*. O cliente nunca pode gravá-lo diretamente. Sempre que o nome do resumo for alterado, a mesma operação transacional ou um gatilho de propagação controlado pelo servidor deve atualizar os documentos *Bem_Patrimonial* associados. Na aprovação da requisição de edição descrita na seção 10.2.5, *novo_nome* representa uma proposta de alteração do nome do resumo; o servidor atualiza primeiro a fonte de verdade e, em seguida, sincroniza *nome_equipamento*.

5.4 *Composicao_Reagente* como sub-coleção

A relação N:N *Especificacao_Reagente* $\leftrightarrow$ *Substancia_Quimica* não precisa de uma coleção de nível superior no Firestore: como é sempre consultada a partir de uma especificação já conhecida (nunca “todas as composições do banco”), o caminho natural é uma sub-coleção *Especificacao_Reagente/{id}/Composicao* (ou um *array* de mapas embutido no próprio documento, se o número de substâncias por especificação for pequeno e não crescer sem limite). A constraint *UNIQUE(id_especificacao_reagente, id_substancia_quimica)* da seção 4.16, nesse caso, precisa ser reforçada na *Cloud Function* de escrita (checar duplicidade antes de gravar), já que o Firestore não tem *unique constraints* declarativas.

5.5 Tabelas “Materialized” são coleções próprias, não *views* do SGBD

As tabelas descritas na seção *Materialized (Views)* não são *materialized views* no sentido de PostgreSQL (não existe *REFRESH MATERIALIZED VIEW* no Firestore). Cada uma delas é implementada como sua própria coleção Firestore, escrita por *Cloud Functions* – gatilhos de escrita (*onDocumentWritten*) para contadores que reagem a um evento pontual (ex.: *Lote_Materializado*), ou funções agendadas (*onSchedule*) para os resumos diários/mensais. Ver pseudo-código na seção *Tecnologia e Relatórios*.

5.6 Unicidade de *numero_patrimonio_proposto* no Firestore

O PostgreSQL implementa a regra RF10b via índice único parcial *UNIQUE(numero_patrimonio_proposto WHERE status = 'pendente')*, impedindo duas requisições simultâneas para a mesma plaqueta enquanto o bem patrimonial ainda não existe. No Firestore não há índice parcial nem *unique constraint* nativo, então a garantia é reimplementada na *Cloud Function* de criação da requisição de adição.

A função de criação consulta a coleção `Requisicao_Adicao_Bem_Patrimonial` filtrando por `numero_patrimonio_proposto == X` e `status == 'pendente'`, com `limit(1)`. Se encontrar algum documento, lança um `HttpError` de *failed-precondition* e não cria nova requisição. Caso contrário, persiste a requisição com `status = 'pendente'` e `feita_em` baseado em `serverTimestamp()`. A consulta e a escrita são encapsuladas em uma transação para evitar condição de corrida entre dois clientes tentando criar requisições para o mesmo número de patrimônio.

5.7 Mapeamento Completo de Coleções e Sub-coleções Firestore

i Como ler esta tabela

Cada linha descreve uma entidade 3FN da Seção 4, a decisão de estrutura no Firestore (coleção raiz, sub-coleção ou array de maps embutido) e a justificativa. Campos FK referenciados em queries de listagem ou relatório que exigiriam um JOIN são obrigatoriamente denormalizados no documento Firestore; o símbolo **Denorm:** lista esses campos. A consistência de todos os campos denormalizados deve ser garantida por `onDocumentUpdated` na Cloud Function da entidade de origem, nunca pelo cliente.

Entidade 3FN	Estrutura Firestore	Justificativa / Denorm
Usuario	Coleção raiz <code>Usuarios</code> (docId = Firebase Auth UID)	Acesso direto por auth; estrutura natural do SDK.
Chefe_Geral	Coleção raiz <code>Chefe_Geral</code> (docId = uid)	Lido por <code>atualizarCustomClaims(uid)</code> para compor o claim <code>roles</code> do token JWT. Papéis não são mais relidos a cada chamada de API.
Gestor_Almojarifado	Coleção raiz <code>Gestor_Almojarifado</code> (docId = uid)	Idem acima.
Gestor_Bens_Patrimoniais	Coleção raiz <code>Gestor_Bens_Patrimoniais</code> (docId = uid)	Idem.

Entidade 3FN	Estrutura Firestore	Justificativa / Denorm
Professor	Coleção raiz Professor (docId = uid)	Idem.
Aluno	Coleção raiz Aluno (docId = uid). Denorm: letra_inicial	Idem. letra_inicial permite filtro de igualdade na busca.
Bolsista	Coleção raiz Bolsista (docId = uid)	Idem.
Gestor_Almojarifado_x_Almojarifado	Coleção raiz	Query bidirecional: “gestores de X” e “almojarifados de Y”.
Almojarifado	Coleção raiz	Poucas dezenas de documentos.
Local	Coleção raiz	Referenciado por múltiplas entidades; campos predio , andar , sala denormalizados em Bem_Patrimonial via trigger.
Resumo_Bem_Patrimonial	Coleção raiz. Denorm: letra_inicial_nome	Agrupador de bens; filtro alfabético por letra inicial.
Bem_Patrimonial	Coleção raiz. Denorm: nome Equipamento , e relatórios. predio , andar , sala , letra_inicial_nome	Entidade central pesquisada diretamente; evita JOINS em listagens
Historico_Bem_Patrimonial	Sub-coleção Bem_Patrimonial/{id}/Historico	Sempre consultado a partir de um bem
Alteracao_Bem_Patrimonial	Campo alteracoes (array de maps) dentro de cada doc de Historico	Poucos campos por alteração; relação 1:N estreita, nunca consultada independentemente.
Requisicao_Edicao_Bem_Patrimonial	Coleção raiz. Denorm: nome Equipamento , status. numero_patrimonio	Listada independentemente nos painéis de gestores por qualquer filtro de

Entidade 3FN	Estrutura Firestore	Justificativa / Denorm
Requisicao_Adicao_Bem_Patrimonial	Coleção raiz	Mesma lógica; query por status e número de patrimônio proposto.
Impressao_Etiqueta_Frasco	Coleção raiz	Poucas dezenas de documentos; necessário para oferecer as três opções do modal de geração.
Contador_Codigo_Frasco	Documento singleton em Contador_Codigo_Frasco/aliasingleton	Lido e incrementado atonicamente junto com a criação do frasco em um frasco/aliasingleton
Substancia_Quimica	Coleção raiz	Catálogo global; raramente alterado.
Resumo_Reagente	Coleção raiz. Denorm: letra_inicial, estado_fisico	Catálogo de reagentes; filtros de igualdade combinados exigem ambos os campos como propriedades do documento.
Especificacao_Reagente	Sub-coleção Resumo_Reagente/{id}/Especificacao	Sempre acessada a partir do resumo; id/Especificacao globalmente.
Composicao_Reagente	Array de maps embutido no documento de Especificacao_Reagente	N pequeno e fixo (poucas substâncias por especificação); não cresce sem limite.
Lote	Coleção raiz. Denorm: id_especificacao_reagente, nome_reagente	Relatórios consultam todos os lotes de um período sem partir de uma especificação conhecida.
Frasco_Reagente	Coleção raiz. Denorm: id_especificacao_reagente, id_resumo_reagente, nome_reagente, unidade_medida, id_almoxarifado	Entidade central; queries cruzadas constantes por almoxarifado, status e reagentes.
Historico_Frasco_Reagente	Coleção raiz (não sub-coleção). Denorm: id_almoxarifado	Jobs diários consultam “todos os históricos do dia” sem partir de um frasco conhecido; sub-coleção impossibilitaria esse padrão.

Entidade 3FN	Estrutura Firestore	Justificativa / Denorm
Emprestimo_Reagente	Coleção raiz. Denorm: id_almoxarifado, nome_reagente, nome_usuario_retirou	Relatórios por almoxarifado e por período; queries de status global (EM_USO).
Materia	Coleção raiz	Catálogo global; raramente alterado.
Professor_x_Materia	Coleção raiz	Query bidirecional: “matérias de X” e “professores de Y”.
Turma	Coleção raiz. Denorm: nome_materia	Acesso direto por <code>codigo_turma</code> ; nome da matéria necessário na listagem sem JOIN.
Aluno_x_Turma	Sub-coleção Turma/{id}/Alunos	Query natural: “alunos desta turma”; contagem de capacidade trivial.
Historico_Alunos_Turma	Sub-coleção Turma/{id}/HistoricoAlunos	Auditoria de inclusão/exclusão sempre na turma .
Post	Sub-coleção Turma/{id}/Posts	Feed cronológico da turma; nunca consultado globalmente.
Comentario	Sub-coleção Posts/{id}/Comentarios	Thread aninhada ao post.
Historico_Posts_Turma	Sub-coleção Posts/{id}/HistoricoPost	Auditoria de edição sempre a partir do post .
Historico_Comentario	Sub-coleção Comentarios/{id}/HistoricoComentario	Auditoria de edição sempre a partir do comentário .
Roteiro_Experimento	Coleção raiz	Compartilhamento cross-professor exige acesso sem partir de uma turma.
Roteiro_Professor_Compartilhado	Coleção raiz	Query bidirecional: “roteiros compartilhados comigo” e “com quem compartilhei este roteiro”.
Convite_Aluno	Coleção raiz	Query por e-mail normalizado e status independentemente de turma.
Notificacao	Sub-coleção Usuarios/{uid}/Notificacoes	Query natural por usuário; funciona naturalmente para usuários multi-role sem necessidade de multiple reads.
Registro_de_Auditoria	Coleção raiz	Log global queryável por tipo de entidade e período.

Entidade 3FN	Estrutura Firestore	Justificativa / Denorm
Lote_Materializado	Coleção raiz (docId = id do lote)	Atualizada por trigger <code>onFrascoCriado/onFrascoRemovido</code> .
Resumo_Almojarifado_Diario	Coleção raiz	Job agendado diário.
Resumo_Reagente_Diario	Coleção raiz	Job agendado diário.
Resumo_Bem_Patrimonial_Diario	Coleção raiz	Job agendado diário.
Atividade_Gestor_Almojarifado_Mensal	Coleção raiz	Job agendado diário (acumulado por mês).
Atividade_Gestor_Bens_Patrimonial_Mensal	Coleção raiz	Job agendado diário (acumulado por mês).

Regra geral de denormalização

Todo campo FK referenciado em queries de listagem ou relatório que exigiria um JOIN deve ser denormalizado no documento Firestore correspondente. A consistência desse campo é mantida por um `onDocumentUpdated` na Cloud Function da entidade de origem, nunca pelo cliente. A ordem de atualização é: primeiro a entidade de origem, depois os documentos dependentes.

5.8 Estratégia de Busca Textual no Firestore

i Por que não existe LIKE no Firestore

O Firestore não oferece busca por substring (sem LIKE, sem `icontains`). A estratégia adotada é: aplicar **filtros obrigatórios de igualdade** no Firestore para reduzir drasticamente o conjunto de documentos retornados, e então aplicar **substring no frontend** sobre os poucos resultados restantes.

5.8.1 Reagentes

O usuário deve obrigatoriamente escolher ao menos um dos filtros: estado físico (**SOLIDO** ou **LIQUIDO**), natureza química (**ORGANICO**, **INORGANICO**, **ELEMENTO** ou **HIBRIDO**), tipo de substância (**PURA** ou **MISTURA**) e/ou letra inicial (campo `letra_inicial` denormalizado). A query Firestore usa esses filtros de igualdade e retorna poucos documentos; o frontend aplica a busca por substring sobre o campo `nome`.

5.8.2 Bens Patrimoniais

Combinação obrigatória de pelo menos um: (Prédio) ou (letra inicial do nome do equipamento) ou (Status). O Firestore filtra com os campos denormalizados `predio`, `andar`, `sala`, `status` e `letra_inicial_nome`. O frontend aplica substring sobre `nome Equipamento` e `numero Patrimônio` nos resultados. Bigramas não serão implementados na V1: o filtro de (Prédio + Andar) já reduz os candidatos suficientemente.

5.8.3 Alunos

Busca via campo denormalizado `letra_inicial` (filtro de igualdade). O frontend aplica substring sobre `nome` nos resultados. A *range query* por prefixo (`nome >= "A"`, `nome < "B"`) é alternativa válida, mas não combina facilmente com outros filtros de igualdade em índice composto no Firestore; o campo `letra_inicial` é preferível.

5.8.4 Professores

Poucos no sistema (dezenas). Estratégia: baixar todos via `onSnapshot` com persistência habilitada e filtrar inteiramente no frontend por substring. Cache local resolve o desempenho.

Listeners com `onSnapshot` e Persistência

O frontend deve usar `onSnapshot` com persistência habilitada (`enableIndexedDbPersistence` ou `enableMultiTabIndexedDbPersistence` conforme o contexto) em todas as coleções listáveis pelo papel do usuário logado. Com isso, o SDK Firestore mantém cache local e trafega apenas deltas do servidor a cada reconexão, reduzindo leituras faturáveis e melhorando a experiência offline. As telas de busca se alimentam primariamente do cache; a sincronização em tempo real é transparente.

6 Materialized (Views)

6.1 Tabela Materialized para Lote

Lote_Materializado

<i>id</i>	SERIAL	PK
<i>id_lote</i>	INTEGER	FK, UNIQUE
<i>qtd_frascos_cadastrados</i>	INTEGER	DEFAULT 0, NOT NULL

i Observação

Corrigido o typo SERAIL → SERIAL. *id_lote* recebeu UNIQUE: esta é uma tabela 1:1 com *Lote* (uma linha por lote), então o par (*id*, *id_lote*) sem essa restrição permitiria duas linhas materializadas divergentes para o mesmo lote. Mantida por trigger/*Cloud Function* a cada frasco cadastrado ou removido daquele lote (ver seção *Tecnologia e Relatórios*).

6.2 Tabela Resumo de Almojarifado - Diário

Resumo_Almojarifado_Diario

<i>id</i>	SERIAL	PK
<i>data</i>	DATE	NOT NULL
<i>id_almojarifado</i>	INTEGER	FK, NOT NULL
<i>qtd_frascos_fechados_no_fim_do_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_frascos_abertos_no_fim_do_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_frascos_emprestados_no_fim_do_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_frascos_cadastrados_fechados_durante_o_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_frascos_cadastrados_abertos_durante_o_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_frascos_emprestados_durante_o_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_frascos_devolvidos_durante_o_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>volume_total_usado_nos_frascos_devolvidos_durante_o_dia</i>	NUMERIC(12,2)	DEFAULT 0, NOT NULL

<i>volume_total_disponivel_ml</i>	NUMERIC(12,2)	DEFAULT 0, NOT NULL
<i>volume_utilizado_no_dia_ml</i>	NUMERIC(12,2)	DEFAULT 0, NOT NULL
<i>massa_total_disponivel_g</i>	NUMERIC(12,2)	DEFAULT 0, NOT NULL
<i>massa_utilizada_no_dia_g</i>	NUMERIC(12,2)	DEFAULT 0, NOT NULL
<i>created_at</i>	TIMESTAMP	NOT NULL
<i>updated_at</i>	TIMESTAMP	NOT NULL

i Separação Física Rigorosa: Massa (*g*) vs Volume (*ml*)

Esta tabela consolida uma distinção única e consistente entre **estoque** (..._no_fim_do_dia, fotografia às 23h59) e **fluxo** (..._durante_o_dia, eventos ocorridos naquele dia). Para eliminar distorções métricas em sólidos com densidade desconhecida ou nula, as colunas agregadas de saldo e consumo são formalmente segregadas nas duas dimensões físicas fundamentais: reagentes líquidos acumulam em *volume_..._ml* e reagentes sólidos acumulam em *massa_..._g*. Nunca se tenta converter gravimetria de sólidos para volume sem densidade aferida.

6.3 Tabela Resumo de Reagente/Solvente - Diário

Resumo_Reagente_Diario

<i>id</i>	SERIAL	PK
<i>data</i>	DATE	NOT NULL
<i>id_resumo_reagente</i>	INTEGER	FK, NOT NULL
<i>id_almoxtarifado</i>	INTEGER	FK, NOT NULL
<i>qtd_frascos_fechados_disponiveis</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_frascos_abertos_disponiveis</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_frascos_em_uso</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_frascos_vazios</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_frascos_descartados_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>volume_total_disponivel_ml</i>	NUMERIC(12,2)	DEFAULT 0, NOT NULL
<i>volume_utilizado_no_dia_ml</i>	NUMERIC(12,2)	DEFAULT 0, NOT NULL
<i>volume_evaporado_no_dia_ml</i>	NUMERIC(12,2)	DEFAULT 0, NOT NULL
<i>massa_total_disponivel_g</i>	NUMERIC(12,2)	DEFAULT 0, NOT NULL
<i>massa_utilizada_no_dia_g</i>	NUMERIC(12,2)	DEFAULT 0, NOT NULL
<i>massa_evaporada_no_dia_g</i>	NUMERIC(12,2)	DEFAULT 0, NOT NULL
<i>created_at</i>	TIMESTAMP	NOT NULL
<i>updated_at</i>	TIMESTAMP	NOT NULL

6.4 Tabela Resumo de Bens Patrimoniais - Diário

Resumo_Bem_Patrimonial_Diario

<i>id</i>	SERIAL	PK
<i>data</i>	DATE	NOT NULL
<i>id_local</i>	INTEGER	FK, NULL
<i>qtd_bens_ativos_no_fim_do_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_bens_inservivel_no_fim_do_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_bens_dado_baixa_no_fim_do_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_bens_cadastrados_durante_o_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_bens_marcados_inservivel_durante_o_dia</i>	INTEGER	
<i>DEFAULT 0, NOT NULL</i>		
<i>qtd_bens_dados_baixa_durante_o_dia</i>	INTEGER	DEFAULT 0, NOT NULL

<i>qtd_requisicoes_abertas_durante_o_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_requisicoes_aprovadas_durante_o_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_requisicoes_rejeitadas_durante_o_dia</i>	INTEGER	DEFAULT 0, NOT NULL
<i>created_at</i>	TIMESTAMP	NOT NULL
<i>updated_at</i>	TIMESTAMP	NOT NULL

6.5 Tabela de Atividade Mensal - Gestor de Almoxarifado

Atividade_Gestor_Almoxarifado_Mensal

<i>id</i>	SERIAL	PK
<i>id_gestor</i>	INTEGER	FK, NOT NULL
<i>ano</i>	INTEGER	NOT NULL
<i>mes</i>	INTEGER	NOT NULL
<i>qtd_reagentes_adicionados</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_frascos_adicionados</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_movimentacoes_registradas</i>	INTEGER	DEFAULT 0, NOT NULL
<i>created_at</i>	TIMESTAMP	NOT NULL
<i>updated_at</i>	TIMESTAMP	NOT NULL

Regra de negócio

UNIQUE(id_gestor, ano, mes).

6.6 Tabela de Atividade Mensal - Gestor de Bens Patrimoniais

Atividade_Gestor_Bens_Patrimoniais_Mensal

<i>id</i>	SERIAL	PK
<i>id_gestor</i>	INTEGER	FK, NOT NULL
<i>ano</i>	INTEGER	NOT NULL
<i>mes</i>	INTEGER	NOT NULL
<i>qtd_bens_adicionados</i>	INTEGER	DEFAULT 0, NOT NULL

<i>qtd_bens_editados</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_requisicoes_aprovadas</i>	INTEGER	DEFAULT 0, NOT NULL
<i>qtd_requisicoes_rejeitadas</i>	INTEGER	DEFAULT 0, NOT NULL
<i>created_at</i>	TIMESTAMP	NOT NULL
<i>updated_at</i>	TIMESTAMP	NOT NULL

Regra de negócio

UNIQUE(id_gestor, ano, mes).

6.7 Catálogo de Métricas Disponíveis por Domínio

- ***Historico_Frasco_Reagente*** (por *tipo*): frascos cadastrados, saídas, entradas, esvaziados, quebrados, descartados, vencidos, ajustes de inventário e evaporação – por dia → alimenta *Resumo_Almojarifado_Diario* e *Resumo_Reagente_Diario*; por gestor/mês → alimenta *Atividade_Gestor_Almojarifado_Mensal*; contagem "agora" → `count()` direto sobre *Frasco_Reagente* filtrado por *vencido = true*.
- ***Emprestimo_Reagente***: empréstimos abertos/devolvidos/atrasados por dia, volume e massa total consumida por dia → alimenta os resumos diários de reagente.
- ***Historico_Bem_Patrimonial* / *Alteracao_Bem_Patrimonial***: bens cadastrados, editados e baixados por dia → alimenta *Resumo_Bem_Patrimonial_Diario*.
- ***Requisicao_Edicao_Bem_Patrimonial* / *Requisicao_Adicao_Bem_Patrimonial***: requisições abertas/aprovadas/rejeitadas por dia → alimenta *Resumo_Bem_Patrimonial_Diario*.
- ***Registro_de_Auditoria***: log genérico entre todas as entidades.

7 Requisitos e Regras de Negócio

7.1 Requisitos funcionais

ID	Requisito
RF01	O sistema deve permitir login por e-mail e senha.
RF02	O sistema deve permitir login com Google através do provedor de autenticação configurado.
RF03	O sistema deve permitir recuperação de senha.

RF04	O sistema deve permitir que um usuário possua múltiplos papéis.
RF05	O sistema deve permitir alternância de papel ativo no dashboard quando houver mais de um papel autorizado.
RF06	O sistema deve permitir cadastro e manutenção de bens patrimoniais conforme permissões.
RF07	O sistema deve manter histórico auditável de alterações patrimoniais.
RF08	O sistema deve permitir requisição de adição de patrimônio.
RF09	O sistema deve permitir requisição de edição de patrimônio.
RF10	O sistema deve impedir mais de uma requisição de edição pendente para o mesmo bem.
RF11	O sistema deve permitir aprovação ou rejeição de requisições.
RF12	O sistema deve permitir registro da baixa de um bem após o procedimento institucional.
RF13	O sistema deve permitir cadastro de almoxarifados.
RF14	O sistema deve calcular pesos e volumes de frascos de reagentes utilizando a densidade.
RF15	O sistema deve registrar empréstimo, devolução, descarte, esvaziamento e quebra de frascos.
RF16	O sistema deve permitir consulta por nome, número de patrimônio, status, local e demais filtros aplicáveis.
RF17	O sistema deve permitir criação de turmas com capacidade máxima estabelecida.
RF18	O sistema deve permitir entrada de alunos em turmas por código ou e-mail.
RF19	O sistema deve permitir criação de posts em turmas pelo professor responsável.
RF20	O sistema deve permitir comentários em posts conforme regras de acesso.
RF21	O sistema deve permitir upload de roteiros em PDF.
RF22	O sistema deve permitir compartilhamento de roteiros entre professores.
RF23	O sistema deve permitir vínculo de roteiro a turma por meio de post.
RF24	O sistema deve gerar relatórios mensais e por período, incluindo consumo em mL e massa em g.
RF25	O sistema deve manter dados históricos e de auditoria sem apagar fatos relevantes.

7.2 Regras de Negócio Gerais

7.2.1 Status de um frasco de reagente

Regra de negócio

quando um frasco de reagente é cadastrado como em quarentena, deve ser obrigatório escrever o detalhe do status explicando o motivo dele estar em quarentena. Um frasco no estado de quarentena, pode ser descartado ou voltar ao estado de disponível (FechadoDisponível ou AbertoDisponível)

7.2.2 Seleção do tipo em Resumo de Reagente

Regra de negócio

Ao cadastrar uma *Especificacao_Reagente*, o estado físico da especificação (SOLIDO ou LIQUIDO) determina quais campos operacionais de frasco serão apresentados. O tipo de substância (PURA ou MISTURA) determina as propriedades químicas obrigatórias, conforme a matriz de obrigatoriedade. O estado físico pertence à especificação e não deve ser inferido ou gravado como propriedade independente do resumo na modelagem 3FN.

7.2.3 Excluindo um Aluno de uma turma

Regra de negócio

Quando o professor exclui um aluno de uma turma, ele não apaga o aluno d sistema. Apenas o retira da turma. Seu histórico nos posts é mantido. Ou seja, o alunos que ainda estão na turma conseguem ver os comentários dele e também seu nome e foto de perfil nos comentários dele.

7.2.4 Chefe Geral

Regra de negócio

O sistema pode ter mais de um chefe geral, isso não altera o sistema.

7.2.5 O responsável pelo bem patrimonial não é um Usuário do sistema LCQUI

Regra de negócio

Por iss que "*nome_responsavel_sei*"é um simples texto.

7.2.6 Historico de Posts na Turma

Regra de negócio

A entidade desse Histórico não tem o id de quem editou porque apenas o Professor da turma específica pode fazer e editar um Post. O *id_turma* não é armazenado nesta entidade 3FN porque pode ser obtido por *Historico_Posts_Turma.id_post* → *Post.id_turma*; se a consulta direta por turma for necessária no Firestore, esse vínculo poderá ser denormalizado na coleção de histórico.

7.2.7 Não compartilhar o mesmo roteiro 2 vezes com um professor

Regra de negócio

Na tabela: *Roteiro Professor Compartilhado*
UNIQUE(id_roteiro_experimento, id_professor)

7.2.8 Densidade e concentração na especificação do reagente

Regra de negócio

A densidade é uma propriedade da *Especificacao_Reagente*, e não do *Resumo_Reagente*. Ela deve ser informada quando obrigatória para a especificação e, após a confirmação cadastral, não pode ser alterada diretamente, pois participa dos cálculos de volume e consumo dos frascos. A concentração pertence à composição da especificação. Duas especificações com o mesmo nome comercial podem ter densidade, concentração, fabricante ou código de produto diferentes e, portanto, são especificações distintas sob o mesmo resumo quando ainda representam o mesmo item químico agrupador; não se deve criar um novo *Resumo_Reagente* apenas por haver diferença de concentração ou densidade.

7.2.9 Regra para código do frasco de Reagente

Regra de negócio

O código do frasco de reagente, mais especificamente "*codigo_frasco*" é gerado pelo próprio sistema e é único, serve para identificar o frasco de reagente na saída e na entrada. Deve haver uma pequena explicação no front-end dizendo que o código é gerado pelo sistema

7.2.10 Regra em Notificação para Professor

Regra de negócio

O `id_turma` só é NULL quando o tipo não é de comentário. A notificação `DATA_DEVOLUCAO_REAGENTE` é preventiva: deve ser emitida pelo backend quando um empréstimo ainda está `EM_USO` e sua data de devolução é hoje ou amanhã. A detecção é responsabilidade da Cloud Function; o frontend apenas apresenta e pode agrupar visualmente os registros em “atrasados”, “vence hoje” e “vence amanhã”. O frontend não decide se uma notificação deve ser criada.

O job pode usar uma única consulta por `status = EM_USO` e `data_devolucao_prevista <= fim_de_amanha`, mas deve classificar cada empréstimo no servidor antes de criar a notificação, separando a janela de atraso da janela preventiva e aplicando idempotência para não gerar a mesma notificação repetidamente.

Deve aparecer na interface:

- COMENTÁRIO: x novos comentários de Fulano de Tal na turma y;
- REQUISICAO_BEM: Nova atualização em sua requisição de Bem patrimonial;
- DATA_DEVOLUCAO_REAGENTE: a data de devolução de um ou mais frasco(s) está expirando [amanhã, hoje];
- ROTEIRO_COMPARTILHADO: Você tem x novo(s) roteiro(s) compartilhado(s) com você.

7.2.11 Regra para convidar um aluno que não está no sistema

Regra de negócio

Quando o aluno é convidado sem especificar uma turma, `id_turma` é NULL em *Convite para Aluno*. Quando o professor vai convidar o aluno, ele pode ou não especificar uma turma, mas ao enviar um convite, tem que aparecer uma confirmação no caso de estar convidando um aluno sem uma turma para ele.

7.2.12 Regra para registrar um patrimônio como dado baixa

Regra de negócio

É preciso fazer upload do documento pdf que comprove que o bem patrimonial foi dado baixa no sistema da UENF

7.2.13 Cálculo de Volume e Peso de Reagentes (Padrão de Laboratório)

Controle de Volume via Pesagem

O sistema adotará a pesagem como método principal de controle de volume dos frascos, utilizando a fórmula $Volume = \frac{Peso_{total} - Peso_{vazio}}{Densidade}$.

1. Cadastro de Frasco Fechado:

O usuário informa o Peso Total (frasco + reagente) medido na balança e o Volume Nominal (ex: 100mL impresso no rótulo). O sistema calcula e salva o peso do recipiente vazio:

$$Peso_{vazio} = Peso_{total} - (Volume_{nominal} \times Densidade)$$

2. Cadastro de Frasco já Aberto (Opção A - Conhece o peso do recipiente):

O usuário informa o Peso do Frasco Vazio e o Peso Total atual na balança. O sistema calcula o volume atual:

$$Volume_{atual} = \frac{Peso_{total} - Peso_{vazio}}{Densidade}$$

3. Cadastro de Frasco já Aberto (Opção B - Conhece o volume atual):

O usuário informa o Volume Atual visual e o Peso Total na balança. O sistema calcula o peso do frasco vazio para uso futuro:

$$Peso_{vazio} = Peso_{total} - (Volume_{atual} \times Densidade)$$

4. Empréstimo e Devolução:

No empréstimo, registra-se o *peso_saida*. Na devolução, registra-se o *peso_retorno*. O consumo é documentado precisamente na tabela *Emprestimo_Reagente* como:

$$Volume_{utilizado} = \frac{Peso_{saida} - Peso_{retorno}}{Densidade}$$

7.2.14 Exclusividade na retirada de reagentes

Regra de negócio

Apenas os usuários cadastrados como *Professor* ou *Bolsista* têm permissão para retirar reagentes do almoxarifado (o *Emprestimo_Reagente.id_usuario_retirou* deve corresponder a um *Usuario* com uma dessas duas linhas associadas). Demais Alunos (mesmo de iniciação científica ou TCC, sem o papel de Bolsista) não possuem autorização no sistema para esta ação. Esta regra foi corrigida para não contradizer a seção *Bolsista* (3.6), que descreve explicitamente o gestor de reagentes fazendo empréstimos para usuários Bolsista.

7.2.15 Usuário Multi-role, exceto Chefe Geral

Regra de negócio

Uma conta que possui o papel *Chefe Geral* não pode possuir nenhum outro papel. Se a mesma pessoa exercer, na instituição, a função de Chefe Geral e também uma função representada por outro papel do sistema, deverão existir duas contas independentes, cada uma com identidade e permissões próprias.

7.2.16 Usuário Multi-role, Aluno não pode ser professor

Regra de negócio

O usuário que é aluno não pode ser professor.

7.2.17 Usuário Multi-role, Quais multi papeis podem existir

Regra de negócio

Pode haver:

Professor que é Gestor de Bens Patrimoniais.

Professor que é Gestor de Almoxarifado

Professor que é Gestor de Bens Patrimoniais e Gestor de Almoxarifado

Aluno que é Bolsista

Aluno que é Bolsista e Gestor de Bens Patrimoniais

Aluno que é Gestor de Almoxarifado

Aluno que é Gestor de Bens Patrimoniais

Aluno que é Gestor de Bens Patrimoniais e Gestor de Almoxarifado

O Usuário Aluno que também é Bolsista não pode ser Gestor de Almoxarifado. Se, na vida real, um aluno é bolsista e também precisa ser Gestor de Almoxarifado, devem ser criadas duas contas para ele no sistema.

No caso do Aluno que exerce algum papel de Gestor (ou ambos): o aluno pode editar bens patrimoniais e/ou reagentes, de acordo com seus papéis respectivos, porque também tem outro papel além de Aluno.

7.2.18 Aluno que é bolsista

Regra de negócio

Apenas o chefe geral consegue colocar um aluno como bolsista.

7.2.19 Status, vencimento e descarte de frascos

Regra de negócio

- Um frasco **VAZIO** ou **QUEBRADO** deve ser exibido como “Pendente de Descarte”, salvo se já estiver **DESCARTADO**.
- Um frasco **VENCIDO** não é automaticamente descartado e também não é automaticamente bloqueado para uso. Para frasco vencido, o sistema mantém a decisão explícita: `em_quarentena = true`, ou `uso_vencido_autorizado = true`, ou situação de *PENDENTE DE DESCARTE* com `uso_vencido_autorizado = false`.
- Em cadastro, quando a validade efetiva já tiver expirado no instante da operação, o gestor é obrigado a escolher um desses três destinos: **QUARENTENA**, **PENDENTE_DE_DESCARTE** ou **DISPONIVEL** com uso vencido explicitamente autorizado para práticas didáticas e demonstrações.
- O frasco em quarentena não pode ser retirado; o frasco pendente de descarte não pode ser retirado; o frasco vencido com uso explicitamente autorizado pode ser retirado, mas a interface deve alertar transparentemente antes da confirmação do empréstimo.
- Se um frasco vencer enquanto estiver emprestado, o empréstimo continua válido. Na devolução, o servidor deve recalcular o vencimento com a hora atual e permitir ao gestor registrar o destino pós-validade.

7.3 Matriz formal de obrigatoriedade e dependência de campos

Entidade	Condição	Obrigatoriedade
Resumo Reagente	sempre	nome, tipo, natureza e limiar de escassez positivo, expresso em quantidade de frascos.
Resumo Reagente	requer pesagem frequente	frequência em dias positiva.
Especificação	SOLIDO	unidade = g; densidade opcional.
Especificação	LIQUIDO	unidade = ml; densidade positiva obrigatória.
Especificação	MISTURA	pelo menos uma composição válida.
Frasco	QUARENTENA	detalhe do status obrigatório.
Frasco	validade conhecida	validade_fechado e validade_efetiva devem ser consistentes; se já expirada, exige decisão de destino.
Frasco	validade expirada	exige destino QUARENTENA, PEN-DENTE_DE_DESCARTE ou DISPONIVEL com uso_vencido_autorizado.
Frasco	FECHADO	capacidade nominal e peso total; peso vazio calculado.
Frasco	abertura	data_abertura e validade_efetiva calculadas no servidor; aplicar mínimo entre validade fechada e prazo após abertura quando ambos existirem.
Frasco	ABERTO	peso total e dados de estimativa conforme modalidade escolhida.
Patrimônio	sempre	id_resumo, numero_patrimonio, estado_conservacao, id_local, photo_url, nome_responsavel_sei e status.
Patrimônio	Ja_dado_baixa	documento comprobatório obrigatório.
Requisição edição	criação	motivo e ao menos um campo proposto.
Requisição	aprovada/rejeitada	respondida_em, respondente e justificativa obrigatórios.
Convite	turma definida	e-mail normalizado + turma.
Convite	sem turma	e-mail normalizado +

7.4 Matriz de combinações Multi-Role

As únicas combinações permitidas para uma mesma conta são:

- Professor + Gestor de Bens Patrimoniais;
- Professor + Gestor de Almoxarifado;
- Professor + Gestor de Bens Patrimoniais + Gestor de Almoxarifado;
- Aluno + Bolsista;
- Aluno + Bolsista + Gestor de Bens Patrimoniais;
- Aluno + Gestor de Almoxarifado;
- Aluno + Gestor de Bens Patrimoniais;
- Aluno + Gestor de Bens Patrimoniais + Gestor de Almoxarifado.

Não são permitidos Chefe Geral combinado com qualquer outro papel, Aluno + Professor, nem Aluno + Bolsista + Gestor de Almoxarifado. A concessão ou remoção de papéis deve ser transacional e auditada.

7.5 Fluxo obrigatório para mutações críticas

Toda operação que altera dados relevantes segue: autenticar, resolver papéis, validar escopo, reler estado atual no servidor, validar transição e constraints, calcular derivados, persistir atômicamente quando necessário, registrar histórico/auditoria e disparar notificações/materializações. O frontend nunca é a autoridade final.

8 Descrição das telas (Dashboards)

8.1 Header em comum em todos os papeis

```
|-----|
|Olá, [Nome completo do usuário] [foto de perfil]|
|-----|
```

Quando se clica na foto de perfil, aparece um modal que se arrasta da direita. Nele é possível deslogar e ir para "Perfil de usuário", que por sua vez abre um modal com as informações do usuário e botão para editar (foto, nome, etc.).

8.2 Header quando o usuário é Multi-Role

```
|-----|
|Olá, [Nome]  [Botão Papel 1] [Botão Papel 2]          [foto de perfil]|
|-----|
```

Alterar os botões de papel no header modifica instantaneamente a página toda e o painel lateral para as funcionalidades daquele papel selecionado (ex: de Aluno para Gestor de Almoxarifado).

8.3 Dashboard-Chefe Geral

O Chefe Geral tem uma div *full-width* logo abaixo do header principal, alterando conforme a aba selecionada no painel esquerdo:

8.3.1 Aba: Almoxarifados

Antes da área principal de busca, ele verá uma div *full-width* com:

- Um botão "Mais" que ao clicar terá as opções:
 - Novo Almoxarifado: Abre um modal para criar um almoxarifado.
 - Novo Gestor de Almoxarifado: Abre um modal em que ele pode escolher: transformar um aluno em gestor de almoxarifado ou criar um Usuário que é Gestor de almoxarifado.
 - **Nova Matéria:** Abre modal com campos Nome e Código da Matéria. O código deve ser único (validado na Cloud Function).
- Um botão para "Gerenciar Gestores de Almoxarifado". Esse botão abre um novo modal que:
 - Permite pesquisar e selecionar Gestores. Ao selecionar, aparece o histórico dele nós últimos 30 dias com: (Reagentes e frascos adicionados, movimentações registradas), caso ele queira de um mês específico, terá um modal para escolher o ano e o mês.
- Um botão de "Relatórios" que abre um modal com as opções para Relatórios em PDF dos almoxarifados.

Abaixo, haverá uma aba de busca onde pode-se filtrar por almoxarifado, reagente (ou ambos) e pesquisar livremente um reagente pelo nome. Ao clicar em um reagente, abre-se uma tela de detalhes: O filtro padrão da tela de detalhes é o estado atual, mas pode ser alterado para um período específico. Ele exibe: Todas as propriedades em Resumo de reagente, inclusive as materializadas (Materialized View).

8.3.2 Aba: Bens Patrimoniais

Antes da aba principal de busca, ele verá uma div *full-width* com:

- Um botão "Mais" com as opções:
 - Novo Gestor de Bens Patrimoniais.
- Um botão para "Gerenciar Gestores de Bens Patrimoniais", abrindo um modal que:
 - Permite pesquisar (por nome) um gestor. Ao selecionar, visualiza-se o histórico dos últimos 30 dias, ou um mês e ano específico que for escolhido (requisições aprovadas/rejeitadas, bens adicionados, edições feitas).
- Um botão de "Relatórios" para gerar os PDFs de bens patrimoniais (mensal, por prédio, geral).

Abaixo, a aba de busca permite filtrar por responsável do bem, estado de conservação, e pesquisa livre pelo número/nome do bem. Ao clicar em um patrimônio, abre-se o detalhamento: Mostra o estado atual, quantos itens existem daquele tipo cadastrados, a divisão por status (Ativo, Inservível, Baixado) e a lista completa desses itens exibindo o número de patrimônio e o respectivo responsável logado no Sistema da UENF.

Ao clicar em um número de patrimônio específico, abre um modal com as propriedades desse patrimônio, ao mudar o status dele para já dado baixa, é preciso fazer o upload do documento pdf que comprove isso.

8.3.3 Aba: Professores

Antes da aba de busca, uma div *full-width* com:

- Um botão para "Novo Professor".

Na área principal ele pode pesquisar Professores pelo nome, filtrar por Matéria lecionada, Prédio, ou número da sala (inclusive combinando múltiplos filtros). Ao clicar em um professor, abre-se um modal para gerenciar a conta daquele docente, ver seu histórico de ações e a lista de turmas que ele administra.

8.3.4 Aba: Alunos

Antes da aba de busca, uma div *full-width* com:

- Um botão para "Novo Aluno".

Na área principal ele pesquisa Aluno pelo nome, podendo filtrar por Matéria que cursa ou número de matrícula. Ao clicar no Aluno, abre-se um modal de gerenciamento exibindo seu histórico e as turmas em que está matriculado.

8.4 Dashboard-Professor

Para o papel de Professor, a dashboard foca em organização acadêmica e gestão de recursos em uso:

- **Painel Lateral:** Fica do lado esquerdo, podendo ser expandido ou mantido em modo compacto. Contém dois grupos:
 - **Grupo 1 (Visualizar):** Opções para acessar a área de "Reagentes" e "Bens Patrimoniais".
 - **Grupo 2 (Turmas):** Lista navegável de todas as turmas ativas dele. Abaixo da lista, um botão "Turmas Arquivadas" (abre um modal com as turmas passadas exibindo Nome, Ano/Semestre e botão para Desarquivar). Ao desarquivar, a turma volta para a lista de turmas tanto do professor, quanto dos alunos que fazem parte dela.

Na aba principal, abaixo do header em comum, ele verá uma div *full-width* com:

- **Notificações:** Botão com ícone de sino que abre um modal *dropdown* alertando sobre requisições de bens respondidas e sobre reagentes emprestados cujo prazo está atrasado, vence hoje ou vence amanhã. Inclui um botão "Limpar tudo".
- **Botão Mais (+):** Abre *dropdown* com: Nova Turma, Novo Aluno (bloqueado se não tiver turmas criadas), Novo Roteiro de Experimento, Novo Pedido de Adição de Bem Patrimonial (se o usuário selecionar "Novo Local" dentro do pedido, um modal sobreposto exigirá o cadastro prévio do Prédio/Sala), e **Nova Matéria** (abre modal simples com nome e código; código deve ser único, validado pelo backend).
- **Gerenciar Roteiros:** Abre um modal com duas abas ("Meus" e "Compartilhados comigo"). Permite abrir para visualização ou baixar os arquivos em PDF.
- **Minhas Requisições:** Modal para acompanhar os pedidos feitos de adição/edição de Bens Patrimoniais (filtrando por Pendente, Aprovada, Rejeitada).
- **Meus Reagentes:** Modal de acompanhamento onde o professor visualiza os reagentes atualmente sob sua posse (ordenados pela data de devolução) e o histórico de uso (Padrão: últimos 30 dias). Ao clicar num reagente histórico, ele visualiza exatamente o volume (mL) que ele consumiu nesse período.

Área Central (Feed da Turma): Ao acessar a dashboard inicialmente, uma mensagem dirá: "*Selecione uma turma ao lado para começarmos*". Ao clicar em uma turma no painel lateral, a área central exibe o Feed dessa disciplina. Existe um bloco

fixo superior para criar novos posts (podendo anexar PDFs novos ou selecioná-los da sua biblioteca/compartilhados). O feed lista os posts em ordem cronológica e cada post contém a área de comentários integrada para interação direta com os alunos.

8.5 Dashboard-Gestor de Almoxarifado

Para o perfil de Gestor de Almoxarifado, a interface é desenhada para a velocidade de atendimento de balcão e auditoria rigorosa de insumos:

- **Painel Lateral:** Grupos de navegação para "Estoque"(Reagentes e Frascos), "Movimentações"(Empréstimos/Retornos), e "Descartes/Alertas".

Na aba principal, uma div *full-width* com botões rápidos essenciais:

- **Alertas de Gestor (Notificações):** Sino de avisos exclusivo exibindo pendências críticas: frascos declarados como vazios, vencidos ou quebrados, ou seja, aguardando mudança para o estado DESCARTADO, e reagentes que atingiram nível de escassez no estoque, frascos que precisam de pesagem de rotina, frascos que estão em quarentena.

- **Botão Mais (+):**

- Novo Reagente -> Abre modal exigindo Nome,CAS, Concentração, grau de pureza(em %), a quantidade em que ele é considerado escasso para alertas futuros, fórmula química , se é controlado pela pf ou pela eb, a classe de inflamabilidade, link da ficha de segurança, tipo (sólido ou líquido: pede densidade, pergunta se requer pesagem frequente, se sim, pede a frequencia em dias). Tem um seletor para escolher entre sólido e líquido (em temperatura ambiente):

- * Para tipo sólido -> Sempre que for adicionar um frasco -> o tipo de medida é gramas (g) (setado pelo sistema)

- * Para tipo líquido -> Sempre que for adicionar um frasco -> o tipo de medida é mililitros (ml) (setado pelo sistema)

Tem um seletor para a Natureza do Reagente: Orgânico, Inorgânico, Elemento ou Híbrido

- Adicionar Frasco: abre um modal para escolher o reagente a ser adicionado, o reagente pode ser pesquisado, por: nome, cas e formula química (deve ter um seletor para escolher, isso serve para não precisar pesquisar em todas as categorias de uma vez). Após selecionar o reagente a ser adicionado, fecha o modal de escolher reagente e passa a preencher o formulário para adicionar o frasco de reagente, o formulário deve conter: Selecionar o almoxarifado em

que esse frasco vai ficar;

Escrever o detalhamento do local (como: Segunda prateleira, na direita, etc)

O número do lote em que esse frasco pertence.

Pergunta o status dele: Fechado, Aberto, Quarentena (em caso de quarente, aparece um campo em texto para ser escrito explicando o motivo de estar em quarentena) O campo "tipo" em Resumo de reagente que determina os campos a serem pedidos abaixo (sólido ou líquido)

líquido ->

- * Se é do tipo fechado, pede apenas a volume do líquido e o peso total com o frasco. (calcula o peso do frasco, mostra o peso do frasco mas não pode ser editado.)
- * Se é do tipo Aberto, pede o peso do frasco, ...

sólido -> pede peso

- **Ações de Bancada (Destacados no centro):**

- **Registrar Retirada:** Modal para escanear/buscar o frasco, selecionar o usuário retirante (Professor ou Bolsista, conforme as regras de autorização) e anotar o Peso Inicial.
- **Registrar Devolução:** Modal crucial. O gestor informa o frasco retornado e insere o **Peso de Retorno** da balança. O sistema faz a diferença, aplica a densidade (se for líquido), e mostra em negrito o *Volume Utilizado em mL*, ou então o *Peso Utilizado em gramas*, atualizando medida no frasco e seus status automaticamente antes de fechar o modal.

- **Relatórios:** Acesso à geração de PDFs mensais.

Área Central de Busca: Tabela geral pesquisável por nome do reagente, status do frasco ou almoxarifado físico. Ao clicar no reagente, o modal apresenta o dashboard do químico: volume total disponível, gráfico de consumo acumulado, e lista dos frascos com localização exata e status atual.

8.6 Dashboard-Gestor de Bens Patrimoniais

Para o Gestor de Bens Patrimoniais, o foco é em auditoria, análise comparativa e suporte processual institucional:

- **Painel Lateral:** Grupos divididos em "Patrimônio Geral", "Requisições de Professores" e "Alertas importantes" (em alertas ele vai ter os patrimônios que estão em estado inservível para poder dar baixa- isso é externo ao sistema.)

Na div *full-width* superior:

- **Notificações:** Alerta sempre que um professor abrir uma requisição de adição de bem novo ou alteração do estado de um bem existente.
- **Botão Mais (+):** Adição direta de bens (ignorando requisição) e cadastro de novas descrições/resumos de itens ou Gestores (se tiver permissão).
- **Analisar Requisições (Destaque):** Abre uma interface de tela dividida (Split-screen). Do lado esquerdo, os dados e foto do bem como estão atualmente no banco de dados. Do lado direito, os novos dados propostos pelo professor destacado em cores (ex: Estado de Conservação passando de Bom para Ruim). No rodapé, botões massivos de "Aprovar Alteração" ou "Rejeitar" (exigindo campo de texto com justificativa).
- **Gerar Relatórios:** Modal para compilação e download de PDF. Foco em permitir gerar o relatório de "Baixa de Bens Inservíveis" (contendo responsáveis) para anexar diretamente no processo interno (SEI) da Universidade.

Área Central de Busca: Pesquisa altamente filtrável (Prédio, Andar, Sala, Nome, Responsável, Status). O clique no bem patrimonial exibe, além das propriedades atuais do equipamento (foto, conservação, etc.), um histórico de auditoria vertical, listando quem modificou, a hora e quais campos foram alterados ao longo do tempo. Também é possível modificar manualmente um equipamento para "Ja_dado_baixa" caso o trâmite no SEI tenha finalizado.

8.7 Dashboard-Aluno

Para o perfil de Aluno, a interface remove completamente as complexidades de gestão, focando apenas no acesso aos materiais acadêmicos:

- **Painel Lateral:** Um visual limpo apresentando a lista de "Minhas Turmas". No rodapé desse painel, um input text estilizado e claro dizendo "Código da Turma" com um botão "Entrar", permitindo vínculo imediato a uma nova disciplina sem modais complexos. Quando o aluno também for um bolsista, ele vai ter o mesmo grupo 1 que o professor: (Visualizar), mas apenas com a opção "Reagentes".

Na aba principal, abaixo do header:

- **Notificações:** Sino simples indicando novos roteiros de práticas postados, ou professores que responderam às dúvidas deixadas nos comentários.

Área Central (Visualização da Disciplina):

- Caso nenhuma turma esteja selecionada no menu lateral, a área exibe um aviso centralizado e amigável: "*Selecione uma turma ao lado para ver as postagens*".
- Ao clicar numa turma, a tela exibe o Feed do aluno. Os posts feitos pelo professor aparecem em forma de *cards* cronológicos.
- Se o post contém um Roteiro de Experimento anexado, um ícone de PDF chamativo aparece ao lado do nome do arquivo com um botão de ação "Baixar"(Download direto).
- Abaixo de cada post, a seção de comentários. O aluno vê um campo simples de *input* para digitar uma dúvida ou observação, que, ao ser enviada, fica visível imediatamente na *thread* daquele post para o professor e demais colegas da sala.

9 Exemplos de fluxos

9.1 Fluxos complementares e critérios de completude

Os fluxos são considerados completos quando descrevem ator, pré-condição, entrada, validação, mudança de estado, efeitos colaterais e resultado. As operações críticas abaixo complementam os cenários de tela já descritos.

9.1.1 Concessão do papel Bolsista

O Chefe Geral seleciona um usuário com papel Aluno. O backend verifica a pré-condição ($Bolsista \implies Aluno$), rejeita a operação se o usuário não for Aluno ou gerar combinação proibida (ex.: Aluno Bolsista atuando como Gestor de Almoxarifado), cria o vínculo de papel na coleção de perfis, registra auditoria e torna a nova permissão de retirada disponível imediatamente.

9.1.2 Aceitação de convite

O aluno apresenta um convite válido. O backend verifica e-mail normalizado, status, expiração e capacidade da turma. A aceitação é idempotente: o vínculo Aluno–Turma é criado uma única vez e o convite deixa de ser pendente. Usuário e papel Aluno são criados apenas quando ainda não existirem.

9.1.3 Arquivamento e desarquivamento de turma

O professor responsável altera o status da turma para Arquivada. Posts, comentários, histórico e vínculos com alunos não são apagados. O evento é auditado e notificações são emitidas quando aplicável. O desarquivamento restaura o status Ativo sem recriar a turma.

9.1.4 Retirada de reagente

O Gestor de Almoxarifado seleciona o frasco e o Professor ou Bolsista retirante. A transação relê o frasco, valida disponibilidade e escopo do gestor, registra o peso de saída, cria o empréstimo EM_USO e muda a disponibilidade para EMPRESTADO. Se o frasco estiver sendo aberto pela primeira vez, sua validade é recalculada antes de aprovar. Se outro processo tiver retirado o frasco antes da confirmação, a transação falha sem criar empréstimo parcial.

9.1.5 Devolução de reagente

O Gestor informa o frasco e o peso de retorno. A transação verifica que o empréstimo está EM_USO ou ATRASADO, recalcula consumo e atraso com dados do servidor (prazo encerrando às 23:59:59.999 do dia previsto), atualiza o empréstimo, grava o evento no histórico e devolve o frasco à disponibilidade adequada ou ao fluxo de descarte/quarentena se vencido ou esgotado.

i Nota sobre o Login

Todos os fluxos detalhados a seguir partem do princípio de que o usuário está deslogado. A primeira etapa padrão é sempre: o usuário acessa a tela de Login do LCQUI, insere suas credenciais (e-mail e senha) ou escolhe o "Login com Google". O sistema exibe um *spinner* (ícone de carregamento giratório) no botão e o redireciona instantaneamente para a sua Dashboard correspondente, aplicando os privilégios do seu Papel (Role).

9.2 Fluxos do usuário Gestor de Bem Patrimonial

9.2.1 Adicionando um bem patrimonial do LCQUI no sistema

Partindo da tela de login, o Gestor acessa o sistema. Caso possua múltiplos papéis, ele clica no botão "Gestor de Bens Patrimoniais" no header. Na dashboard, clica no botão "Mais (+)" e seleciona "Novo Bem Patrimonial". Um modal se sobrepõe à tela. Ele preenche o número do patrimônio, pesquisa um *Resumo_Bem* existente, escolhe o Local (prédio, andar, sala) via menus *dropdown* e anexa a URL da foto. Ao clicar em "Salvar", um *spinner* roda no botão e, em seguida, um *toast* (notificação flutuante) verde surge na tela: "Bem cadastrado com sucesso". O modal fecha e a grid central é recarregada mostrando o novo item.

9.2.2 Baixando a lista de bens inservíveis para dar baixa no SEI

Partindo da tela de login, o Gestor acessa a dashboard e clica no botão "Gerar Relatórios" na parte superior. O modal de relatórios abre. Ele seleciona o tipo "Bens Inservíveis", confirma e clica em "Gerar PDF". O botão muda para o estado *loading*. Após alguns segundos, o navegador exibe a notificação de download concluído ou abre o arquivo em uma nova aba, pronto para ser anexado ao Sistema Eletrônico de Informações (SEI) da UENF.

9.2.3 Baixando o relatório mensal Geral do Mês de Agosto

Após o login, na tela principal, o Gestor clica em "Gerar Relatórios". No modal que se abre, seleciona a aba "Relatório Geral". Nos seletores de data, escolhe "Agosto" e

"2026". Ao clicar em "Gerar", um círculo de progresso é exibido no centro do modal. Finalizado o processo backend, o modal exibe a mensagem "PDF pronto" e inicia o download automaticamente.

9.2.4 Baixando relatório de bens materiais filtrado (Prédio P5, Andar 1)

Partindo da tela de login, o Gestor abre o modal de relatórios e vai até a aba "Personalizado". Ele preenche as datas (15/02/2026 a 16/07/2026) e abre o filtro avançado. Marca a *checkbox* do "Prédio P5" e digita "1" no andar. Clica em "Gerar PDF". Após a barra de carregamento preencher, o sistema entrega o documento formatado apenas com os equipamentos deste andar específico.

9.2.5 Verificando requisições pendentes de professores

Após efetuar o login, a dashboard do Gestor já apresenta no painel central um card de "Painel de Requisições". O botão de notificações (sino) está com uma bolinha vermelha. O Gestor clica na notificação e depois na requisição desejada. A tela faz uma transição para o modo *Split-screen*. Na metade esquerda da tela, ele vê os dados estáticos (antigos) do patrimônio em texto cinza. Na metade direita, ele vê os dados novos sugeridos pelo professor (ex: alteração de sala) destacados em fundo amarelo suave, para chamar a atenção à mudança.

9.2.6 Aprovando uma requisição pendente

Dando continuidade ao fluxo de verificação (após login e abertura da tela de *Split-screen*), o Gestor avalia que a alteração de sala pedida pelo professor está correta. Ele clica no botão verde massivo "Aprovar Alteração" no rodapé. Um ícone de *check* (✓) animado aparece no centro da tela. A requisição desaparece da fila de pendentes e um *toast* de sucesso no canto superior direito confirma que o banco de dados principal foi atualizado.

9.2.7 Alterando um bem patrimonial para: Dado baixa

Partindo da tela de login, o Gestor usa a barra de busca central e digita o número da plaqueta de um equipamento que já passou por todo o processo burocrático no SEI. A tela de detalhes do item se abre. O Gestor clica no botão "Editar Status". Um menu *dropdown* aparece e ele seleciona "Ja_dado_baixa". Ao selecionar esse status, aparece um novo campo obrigatório para fazer upload do documento pdf que comprove seu status. Ao anexar, o sistema libera o botão para salvar a alteração. Porém, antes de salvar, o sistema exibe um modal de atenção: "*Esta ação é irreversível no histórico ativo. Deseja prosseguir?*". O Gestor clica em "Confirmar". Um *toast* azul confirma a

baixa e o item recebe instantaneamente uma etiqueta cinza de inatividade, saindo da listagem de bens ativos.

9.3 Fluxos do usuário Professor

9.3.1 Criando uma turma

Após fazer login, o Professor está na sua dashboard. Ele clica no painel lateral esquerdo em "Turmas" e depois no botão flutuante "+ Nova Turma". Um modal simples é exibido solicitando Nome da Turma e Capacidade. Ele preenche "Química Orgânica I" e "30", e clica em "Salvar". Um *toast* de sucesso é disparado, um novo card desliza aparecendo no painel lateral, e o *codigo_turma* recém-criado pisca rapidamente na tela com um botão de "Copiar para a área de transferência".

9.3.2 Adicionando Alunos em uma turma

Com login feito, o Professor clica no card da turma recém-criada no painel lateral. A área central atualiza para a visão da disciplina. No header, ele clica no botão "Novo Aluno". No modal que surge, ele copia e cola os e-mails dos alunos separados por vírgula e clica em "Enviar Convites". Uma barra de carregamento é mostrada, seguida de um *toast* verde informando que os acessos foram disparados por e-mail.

9.3.3 Criando um Post em uma turma

Após login, o Professor acessa a disciplina clicando nela. No topo do Feed de publicações (área central), ele clica na caixa de texto "Comece um novo post...". O componente se expande. Ele digita as orientações da aula, clica no ícone de "Clipe de Papel" (anexo), o que abre o modal da sua biblioteca de roteiros em PDF. Ele seleciona o "Roteiro_Prática_3.pdf" e clica em "Publicar". A tela rola suavemente para baixo e o novo card da publicação surge com um efeito de *fade-in*, exibindo o texto e a prévia do PDF anexado.

9.3.4 Excluindo um Aluno de uma turma

O Professor loga no sistema e navega até a turma desejada. Ele clica na aba "Membros" logo acima do feed. Uma lista com nome e foto dos alunos aparece. Ao lado do nome do aluno a ser removido, ele clica no ícone vermelho de lixeira. Um modal de dupla confirmação exige que ele clique em "Sim, remover aluno". A linha do aluno desaparece da lista com uma rápida animação de encolhimento (*collapse*) e um *toast* avisa "Aluno removido da turma".

9.3.5 Respondendo um comentário de um aluno

O Professor entra no sistema e nota uma bolinha vermelha piscando no sino de Notificações. Ele clica e vê "*João comentou em um Roteiro*". Ao clicar na notificação, a tela rola automaticamente até a exata postagem no feed. Ele vê a dúvida do aluno, clica na caixa de texto abaixo dizendo "Escreva uma resposta..." e aperta Enter. O comentário dele surge instantaneamente logo abaixo do aluno, com seu nome identificado com a tag "[Professor]".

9.3.6 Pesquisando Reagente Químico para checar disponibilidade

Partindo da tela de login, o Professor clica no atalho "Reagentes" no painel esquerdo. Na grande barra de pesquisa central, ele começa a digitar "Ácido Clori...". O sistema usa *debounce* (pesquisa em tempo real) e já filtra a tabela antes mesmo do Enter. Ele encontra o item, clica sobre a linha e um modal lateral (gaveta) desliza da direita mostrando em verde "*Disponível: 3 frascos fechados, 1 aberto*".

9.3.7 Pesquisando um bem patrimonial pelo Nome

Após login, o Professor vai à área "Bens Patrimoniais" e digita "Microscópio" na busca. A tela é atualizada mostrando uma malha de cards (Grid) com todos os equipamentos que possuem essa palavra. Ele clica em um dos cards e vê a foto e a localização exata daquele Microscópio em tela cheia.

9.3.8 Pesquisando um bem patrimonial pelo número de patrimônio

Professor faz login, acessa a busca de "Bens Patrimoniais" e digita exatamente "987654". O sistema detecta a correspondência exata, pula a exibição da tabela genérica e direciona o professor direto para a página exclusiva de detalhes daquele patrimônio específico.

9.3.9 Requisitando alteração de conservação (de Bom para Ruim)

No passo anterior, ao estar na tela de detalhes do microscópio, o Professor, após logar, percebe que a lente está trincada. Ele clica no botão amarelo "Solicitar Edição". No formulário sobreposto, ele altera o campo *dropdown* "Estado de Conservação" de "Bom" para "Ruim/Inservível", preenche o campo de texto justificando ("Lente principal rachada") e envia. O botão vira um *spinner* rapidamente e um *toast* avisa: "Requisição enviada ao Gestor de Patrimônio".

9.3.10 Upload de Roteiro e compartilhamento com outros professores

Professor acessa a dashboard após login, clica em "Gerenciar Roteiros"(no header central). O modal abre na aba "Meus". Ele clica no ícone de Nuvem para upload, arrasta um PDF do seu computador para a área pontilhada e aguarda a barra de progresso encher (100%). Em seguida, no próprio arquivo recém-subido, ele clica em "Compartilhar". Uma lista de professores do sistema aparece, ele marca 3 caixinhas de seleção e clica no botão "Autorizar Acesso". Um alerta flutuante informa que o roteiro agora está visível nas bibliotecas dos colegas.

9.4 Fluxos do usuário Chefe Geral

9.4.1 Criando um usuário Professor e usuário Gestor de Almoxarifado

O Chefe Geral acessa a plataforma logando com suas credenciais. Na sua dashboard, acessa a aba "Professores" e clica no botão "+ Novo Professor". No modal, insere nome, e-mail institucional e seleciona o Centro (ex: CCT). Ao salvar, um *toast* avisa do envio do convite. Da mesma forma, indo para a aba "Almoxarifados", ele clica em "Mais (+)", depois "Novo Gestor de Almoxarifado", preenche o e-mail, e um novo card de gestor é populado silenciosamente na lista do sistema, notificando o sucesso da operação.

9.5 Fluxos do usuário Aluno

9.5.1 Ingressando em uma Turma

O Aluno acessa o site do LCQUI pela primeira vez, cria a senha (ou entra via Google) e faz o login. A dashboard dele é muito mais limpa e a tela central diz: "*Você ainda não tem turmas*". Ele foca sua atenção no canto inferior esquerdo onde há o input "Código da Turma". Ele digita o *hash* de 6 letras que o professor escreveu no quadro e clica em "Entrar". O botão gira um carregamento, o sistema verifica a capacidade máxima da turma no *backend*, faz o link de tabelas, e instantaneamente um Card colorido com o nome da disciplina desliza para o centro da tela com o feedback: "*Matrícula realizada com sucesso!*".

9.5.2 Baixando um roteiro e realizando um comentário

Logado, o aluno clica na disciplina recém-adicionada. O Feed da turma é carregado na tela. Ele rola a página e acha a postagem do professor com um arquivo. Ele clica no botão vermelho e muito visível de "Baixar PDF". O *download* inicia via navegador. Logo abaixo do mesmo *post*, na seção de interações, ele clica na caixa de texto vazia,

digita "Professor, levaremos óculos de proteção?" e pressiona Enviar. O comentário surge na tela logo após uma animação suave de transição (*fade-in*).

9.6 Fluxos do Usuário Gestor de Almoxarifado

9.6.1 Adicionando um frasco fechado de um reagente existente

Partindo do login, o Gestor de Almoxarifado acessa seu painel, clica no grupo "Estoque", digita o nome de um ácido na busca e clica em "Adicionar Frasco". Ele coloca o frasco lacrado em cima da balança USB (ou lê manualmente), digita no campo "Peso Total" (ex: 610g) e no "Volume Nominal" (ex: 500mL). Ao clicar em "Salvar", o sistema calcula a tara (peso do frasco vazio), registra no banco e o modal se fecha com uma notificação verde "Frasco Fechado registrado no almoxarifado".

9.6.2 Adicionando um frasco já aberto de um novo reagente

Após o login, o Gestor nota que o reagente ainda não existe no sistema. Na dashboard central, ele clica em "Novo Reagente". Preenche as propriedades químicas obrigatórias, com destaque em vermelho para o campo "Densidade" quando líquido. Ao confirmar, o modal avança automaticamente para o passo "Cadastrar Frascos Vinculados". Como o frasco físico está pela metade, ele muda a chave seletora para "Informar Volume Atual (Visual)". Ele digita o peso medido agora, e estima 120mL no input visual. Salva a operação, a tela é atualizada e um aviso diz: "Estoque inicial criado e peso do frasco base arquivado".

9.6.3 Realizando empréstimo (Retirada) de um frasco

Com o login concluído, um Professor ou Bolsista chega no balcão. O Gestor de Almoxarifado clica rapidamente no enorme botão verde "Registrar Retirada" no centro da dashboard. Um modal abre solicitando a leitura do código do frasco ou pesquisa pelo nome. Localizado o item, o Gestor seleciona o usuário autorizado, informa o local de uso e pesa o frasco.

Antes da confirmação, a interface bloqueia frascos em quarentena ou pendentes de descarte. Se o frasco estiver vencido e seu uso vencido tiver sido autorizado, deve aparecer um alerta explícito: "Este frasco venceu em DD/MM/AAAA. O uso vencido está autorizado para fins didáticos. Deseja confirmar o empréstimo mesmo assim?". A confirmação é enviada ao backend, que repete a validação; o frontend não pode ignorar essa regra. Se a operação também registrar a primeira abertura do frasco, a validade efetiva é recalculada no servidor dentro da mesma transação.

9.6.4 Devolvendo um frasco (Cálculo de Consumo)

O professor devolve o vidro à bancada. O Gestor faz login (se necessário) e clica no grande botão “Registrar Devolução”. Busca o item (que está na lista de pendentes). O sistema solicita “Peso de Retorno”, calcula o consumo e verifica a validade efetiva com o relógio do servidor (com prazo normalizado para as 23:59:59.999 do dia previsto). Se o frasco estiver vencido, ou tiver vencido enquanto permaneceu com o professor, a devolução permite registrar a decisão do gestor entre quarentena, descarte ou continuidade de uso didático.

O sistema renderiza na tela em negrito e com fundo destacado: **“Volume utilizado neste empréstimo: 4.5 mL”** ou **“Peso utilizado nesse empréstimo: 22 g”**. Ao clicar em “Finalizar”, um sinal sonoro/visual confirma que a transação de estoque, consumo, histórico e auditoria foram concluídas.

10 Tecnologia e Relatórios (Vercel / Firebase)

10.1 Para relatórios

Após avaliação das restrições do ambiente *serverless* (Vercel e Firebase Functions), a biblioteca escolhida para a geração de relatórios no backend é o ***pdfkit*** em conjunto com o plugin ***pdfkit-table***.

Justificativa da escolha:

- **Baixo consumo de memória:** O *pdfkit* desenha o PDF nativamente, respeitando os limites de memória da Vercel/Firebase.
- **Tabelas dinâmicas:** O *pdfkit-table* resolve a paginação automática de tabelas longas.
- **Fluxo de Armazenamento:** A Cloud Function gera o PDF em um *Buffer* de memória, faz o upload para o **Firebase Storage** e retorna uma URL assinada temporária para download.

10.2 Princípio: nunca confiar no *client-side*

Todas as *Cloud Functions* desta seção seguem a mesma regra: **qualquer dado que o servidor consiga derivar ou verificar por conta própria é recalculado ou reconferido no servidor, nunca aceito como o cliente enviou**. Isso vale, em particular, para o papel do usuário (sempre resolvido a partir do Firestore, nunca do que o *request* diz que o usuário é), para valores calculados como peso/volume/vencimento/atraso (sempre recomputados a partir dos dados brutos e da hora do servidor), para códigos gerados pelo sistema (Regra 6.2.9), e para o estado de disponibilidade de um recurso concorrente (sempre relido dentro de uma transação antes de decidir).

10.2.1 Funções de Apoio (Autorização)

Resolução de Papéis via Custom Claims (zero leituras Firestore)

```
import { onCall, HttpsError } from "firebase-functions/v2/https";
import * as admin from "firebase-admin";

// C1: Papéis lidos diretamente do JWT (Firebase Custom Claims).
// Zero leituras extras no Firestore por chamada de API.
// PREREQUISITO: toda mutação de papel deve chamar atualizarCustomClaims(uid
).
```

```
function resolverPapeisDoToken(auth: {token: Record<string, unknown>} |
  undefined): string[] {
  if (!auth) return [];
  const roles = auth.token["roles"];
  if (!Array.isArray(roles)) return [];
  return roles as string[];
}

// Chamada após TODA concessão ou remoção de papel:
// await atualizarCustomClaims(uid);
// Isso invalida o token atual; o cliente deve renovar com user.getIdToken(
  true).

export async function atualizarCustomClaims(uid: string): Promise<void> {
  const colecoes = ["Chefe_Geral", "Gestor_Almoхарifado", "
    Gestor_Bens_Patrimoniais", "Professor", "Aluno", "Bolsista"];
  const leituras = await Promise.all(
    colecoes.map((c) => admin.firestore().collection(c).doc(uid).get())
  );
  const roles = colecoes.filter((_, i) => leituras[i].exists);
  await admin.auth().setCustomUserClaims(uid, { roles });
}

function validarPermissao(request: {auth?: {uid: string; token: Record<
  string, unknown>}}, papeisPermitidos: string[]): string[] {
  if (!request.auth?.uid) throw new HttpsError("unauthenticated", "Usuário n
    ão autenticado.");
  const papeis = resolverPapeisDoToken(request.auth);
  if (!papeisPermitidos.some((p) => papeis.includes(p))) {
    throw new HttpsError("permission-denied", "Usuário sem papel autorizado
      para esta ação.");
  }
  return papeis;
}

async function validarGestorDoAlmoхарifado(uid: string, auth: {token: Record
  <string, unknown>}, idAlmoхарifado: string): Promise<void> {
  const papeis = resolverPapeisDoToken(auth);
  if (papeis.includes("Chefe_Geral")) return; // Chefe tem escopo global
  if (!papeis.includes("Gestor_Almoхарifado")) {
    throw new HttpsError("permission-denied", "Usuário não é Gestor de
      Almoхарifado.");
  }
}
```

```
}  
// 1 leitura: o vínculo de escopo (não o papel - esse já veio do token)  
const vinculo = await admin.firestore().collection("Gestor_Almoхарifado_x_Almoхарifado")  
  .where("id_gestor_almoхарifado", "==", uid)  
  .where("id_almoхарifado", "==", idAlmoхарifado)  
  .limit(1).get();  
if (vinculo.empty) {  
  throw new HttpsError("permission-denied", "Gestor não está designado para este almoхарifado.");  
}  
}
```

10.2.2 Fluxo de Reagentes: Cadastro, Retirada e Devolução

Cadastro de Frasco Fechado (Regra 6.2.13, item 1)

```
interface CadastroFrascoFechado {  
  idEspecificacaoReagente: string;  
  idAlmoхарifado: string;  
  idLote?: string;  
  pesoTotal: number;  
  volumeNominal: number;  
  validadeFechado?: string;  
  validadeDesconhecida?: boolean;  
  decisaoSeJaVencido?: "QUARENTENA" | "PENDENTE_DE_DESCARTE" | "DISPONIVEL";  
  detalheStatus?: string;  
}  
  
export const cadastrarFrascoFechado = onCall(async (request) => {  
  const dados = request.data as CadastroFrascoFechado;  
  validarPermissao(request, ["Chefe_Geral", "Gestor_Almoхарifado"]);  
  await validarGestorDoAlmoхарifado(request.auth!.uid, request.auth!.token, dados.idAlmoхарifado);  
  
  const especSnap = await admin.firestore().collection("Especificacao_Reagente").doc(dados.idEspecificacaoReagente).get();  
  if (!especSnap.exists) throw new HttpsError("not-found", "Especificação de reagente não encontrada.");  
  const densidade = especSnap.data()!.densidade;  
  const estadoFisico = especSnap.data()!.estado_fisico;
```



```
let pesoVazio: number;
if (estadoFisico === "LIQUIDO") {
  if (!densidade || densidade <= 0) throw new HttpsError("failed-
precondition", "Líquido exige densidade positiva.");
  pesoVazio = dados.pesoTotal - (dados.volumeNominal * densidade);
} else {
  pesoVazio = dados.pesoTotal - dados.volumeNominal; // g
}

if (pesoVazio <= 0) {
  throw new HttpsError("invalid-argument", "Peso total incompatível com o
volume nominal para esta densidade.");
}

const agora = new Date();
const validadeFechado = dados.validadeFechado ? new Date(dados.
validadeFechado) : null;
const validadeEfetiva = dados.validadeDesconhecida ? null :
validadeFechado;
const venceuNoCadastro = Boolean(validadeEfetiva && validadeEfetiva <=
agora);

if (venceuNoCadastro && !dados.decisaoSeJaVencido) {
  throw new HttpsError("failed-precondition", "O frasco está vencido no
cadastro e exige uma decisão do gestor.");
}

if (dados.decisaoSeJaVencido === "QUARENTENA" && !dados.detalheStatus) {
  throw new HttpsError("invalid-argument", "A entrada em quarentena exige
detalhe_status.");
}

const contadorRef = admin.firestore().collection("Contador_Codigo_Frasco").
doc("singleton");
const frascoRef = admin.firestore().collection("Frasco_Reagente").doc();

return admin.firestore().runTransaction(async (tx) => {
  const contadorSnap = await tx.get(contadorRef);
  const proximoNumero = (contadorSnap.data()?.ultimo_codigo_gerado || 0) +
1;
```

```

const codigoFrasco = 'LCQUI-${proximoNumero}';

tx.update(contadorRef, { ultimo_codigo_gerado: proximoNumero });
tx.set(frascoRef, {
  id_almoxarifado: dados.idAlmoxarifado,
  id_lote: dados.idLote ?? null,
  id_especificacao_reagente: dados.idLote ? null : dados.
idEspecificacaoReagente,
  codigo_frasco: codigoFrasco,
  conteudo_nominal: dados.volumeNominal,
  peso_no_cadastrado: dados.pesoTotal,
  peso_atual: dados.pesoTotal,
  peso_frasco_vazio: pesoVazio,
  medida_usada: 0,
  estado_fisico_frasco: "FECHADO",
  disponibilidade: "DISPONIVEL",
  validade_fechado: validadeEfetiva,
  validade_efetiva: validadeEfetiva,
  validade_desconhecida: dados.validadeDesconhecida ?? false,
  vencido: venceuNoCadastro,
  em_quarentena: venceuNoCadastro && dados.decisaoSeJaVencido === "
QUARENTENA",
  uso_vencido_autorizado: venceuNoCadastro && dados.decisaoSeJaVencido
=== "DISPONIVEL",
  detalhe_status: venceuNoCadastro ? (dados.detalheStatus ?? (dados.
decisaoSeJaVencido === "PENDENTE_DE_DESCARTE" ? "Frasco vencido
aguardando processo institucional de descarte." : null)) : null,
  cadastrado_em: admin.firestore.FieldValue.serverTimestamp(),
  cadastrado_por: request.auth!.uid,
});

return { idFrasco: frascoRef.id, codigoFrasco, pesoVazioCalculado:
pesoVazio, venceuNoCadastro };
});
});

```

cadaststrarFrascoAberto com bifurcação SÓLIDO/LÍQUIDO (C5)

```

// C5: modalidade para SÓLIDO usa ESTIMA_MASSA (não ESTIMA_VOLUME)
interface CadastroFrascoAberto {
  idEspecificacaoReagente: string;
  idAlmoxarifado: string;

```

```
idLote?: string;
// LIQUIDO: "CONHECE_TARA" | "ESTIMA_VOLUME"
// SOLIDO: "CONHECE_TARA" | "ESTIMA_MASSA"
modalidade: "CONHECE_TARA" | "ESTIMA_VOLUME" | "ESTIMA_MASSA";
pesoTotalBalanca: number; // sempre leitura de balança em g
pesoFrascoVazioInformado?: number; // Opção A (ambos estados físicos)
volumeAtualEstimado?: number; // Opção B LIQUIDO: volume visual em
    mL
massaAtualEstimada?: number; // Opção B SOLIDO: estimativa em g
validadeAberto?: string;
validadeDesconhecida?: boolean;
decisaoSeJaVencido?: "QUARENTENA" | "PENDENTE_DE_DESCARTE" | "DISPONIVEL";
detalheStatus?: string;
}

export const cadastrarFrascoAberto = onCall(async (request) => {
    const dados = request.data as CadastroFrascoAberto;
    const papeis = validarPermissao(request, ["Chefe_Geral", "Gestor_Almojarifado"]);
    await validarGestorDoAlmojarifado(request.auth!.uid, request.auth!.token,
        dados.idAlmojarifado);

    const especSnap = await admin.firestore().collection("Especificacao_Reagente").doc(dados.idEspecificacaoReagente).get();
    if (!especSnap.exists) throw new HttpsError("not-found", "Especificação não encontrada.");
    const especData = especSnap.data()!;
    const estadoFisico: "SOLIDO" | "LIQUIDO" = especData.estado_fisico;
    const densidade: number | null = especData.densidade ?? null;

    // C5: bifurcação explícita por estado físico
    let pesoVazio: number;
    let conteudoNominal: number; // g para sólidos, mL para líquidos

    if (estadoFisico === "SOLIDO") {
        // Sólidos: tudo em gramas; densidade não é usada para cálculo de volume
        if (dados.modalidade === "CONHECE_TARA") {
            if (dados.pesoFrascoVazioInformado == null)
                throw new HttpsError("invalid-argument", "CONHECE_TARA exige pesoFrascoVazioInformado.");
            pesoVazio = dados.pesoFrascoVazioInformado;
```

```
    conteudoNominal = dados.pesoTotalBalanca - pesoVazio;
} else { // ESTIMA_MASSA
    if (dados.massaAtualEstimada == null)
        throw new HttpsError("invalid-argument", "ESTIMA_MASSA exige massaAtualEstimada.");
    conteudoNominal = dados.massaAtualEstimada;
    pesoVazio = dados.pesoTotalBalanca - conteudoNominal;
}
if (pesoVazio <= 0)
    throw new HttpsError("invalid-argument", "Massa atual maior que o peso total medido na balança.");
} else { // LIQUIDO
    if (!densidade || densidade <= 0)
        throw new HttpsError("failed-precondition", "Líquido exige densidade positiva cadastrada na especificação.");
    if (dados.modalidade === "CONHECE_TARA") {
        if (dados.pesoFrascoVazioInformado == null)
            throw new HttpsError("invalid-argument", "CONHECE_TARA exige pesoFrascoVazioInformado.");
        pesoVazio = dados.pesoFrascoVazioInformado;
        conteudoNominal = (dados.pesoTotalBalanca - pesoVazio) / densidade;
    } else { // ESTIMA_VOLUME
        if (dados.volumeAtualEstimado == null)
            throw new HttpsError("invalid-argument", "ESTIMA_VOLUME exige volumeAtualEstimado.");
        conteudoNominal = dados.volumeAtualEstimado;
        pesoVazio = dados.pesoTotalBalanca - (conteudoNominal * densidade);
    }
    if (conteudoNominal <= 0 || pesoVazio <= 0)
        throw new HttpsError("invalid-argument", "Dados de volume/peso inconsistentes com a densidade.");
}

const agora = new Date();
const validade = dados.validadeAberto ? new Date(dados.validadeAberto) : null;
const validadeEfetiva = dados.validadeDesconhecida ? null : validade;
const jaVencido = Boolean(validadeEfetiva && validadeEfetiva <= agora);

if (jaVencido && !dados.decisaoSeJaVencido)
    throw new HttpsError("failed-precondition", "O frasco está vencido no
```

```

    cadastro e exige uma decisão do gestor.");
if (dados.decisaoSeJaVencido === "QUARENTENA" && !dados.detalheStatus)
    throw new HttpsError("invalid-argument", "A entrada em quarentena exige
    detalhe_status.");

const contadorRef = admin.firestore().collection("Contador_Codigo_Frasco").
    doc("singleton");
const frascoRef = admin.firestore().collection("Frasco_Reagente").doc();

return admin.firestore().runTransaction(async (tx) => {
    const contadorSnap = await tx.get(contadorRef);
    const proximoNumero = (contadorSnap.data()?.ultimo_codigo_gerado || 0) +
    1;
    const codigoFrasco = `LCQUI-${proximoNumero}`;

    tx.update(contadorRef, { ultimo_codigo_gerado: proximoNumero });
    tx.set(frascoRef, {
        id_almoxarifado: dados.idAlmoxarifado,
        id_lote: dados.idLote ?? null,
        id_especificacao_reagente: dados.idLote ? null : dados.
idEspecificacaoReagente,
        codigo_frasco: codigoFrasco,
        conteudo_nominal: conteudoNominal,    // g (sólido) ou mL (líquido)
        peso_no_cadastrado: dados.pesoTotalBalanca,
        peso_atual: dados.pesoTotalBalanca,
        peso_frasco_vazio: pesoVazio,
        medida_usada: 0,
        data_abertura: agora,
        estado_fisico_frasco: "ABERTO",
        disponibilidade: "DISPONIVEL",
        validade_efetiva: validadeEfetiva,
        validade_desconhecida: dados.validadeDesconhecida ?? false,
        vencido: jaVencido,
        em_quarentena: jaVencido && dados.decisaoSeJaVencido === "QUARENTENA",
        uso_vencido_autorizado: jaVencido && dados.decisaoSeJaVencido === "
DISPONIVEL",
        detalhe_status: jaVencido
            ? (dados.detalheStatus ?? (dados.decisaoSeJaVencido === "
PENDENTE_DE_DESCARTE"
            ? "Frasco vencido aguardando processo institucional de descarte.
" : null))

```

```

        : null,
        cadastrado_em: admin.firestore.FieldValue.serverTimestamp(),
        cadastrado_por: request.auth!.uid,
    });

    return { idFrasco: frascoRef.id, codigoFrasco, conteudoNominal,
    pesoVazio, venceuNoCadastro: jaVencido };
  });
});

```

Abertura e cálculo de validade efetiva no servidor

```

function calcularValidadeEfetivaNaAbertura(frasco: any, dataAbertura: Date):
  Date | null {
  if (frasco.validade_desconhecida) return null;

  const validadeFechado = frasco.validade_fechado
    ? (typeof frasco.validade_fechado.toDate === "function"
      ? frasco.validade_fechado.toDate()
      : new Date(frasco.validade_fechado))
    : null;
  const diasDepoisDeAberto = frasco.validade_apos_aberto_dias;
  const validadeDepoisDaAbertura = diasDepoisDeAberto != null
    ? new Date(dataAbertura.getTime() + diasDepoisDeAberto * 24 * 60 * 60 *
    1000)
    : null;

  if (validadeFechado && validadeDepoisDaAbertura) {
    return validadeFechado <= validadeDepoisDaAbertura ? validadeFechado :
    validadeDepoisDaAbertura;
  }
  return validadeFechado ?? validadeDepoisDaAbertura;
}

interface AberturaFrasco {
  idFrasco: string;
  destinoSeVencerNaAbertura?: "QUARENTENA" | "PENDENTE_DE_DESCARTE" | "
  DISPONIVEL";
  detalheStatus?: string;
}

export const registrarAberturaFrasco = onCall(async (request) => {

```

```
const dados = request.data as AberturaFrasco;
await validarPermissao(request.auth?.uid, ["Chefe_Geral", "
  Gestor_Almojarifado"]);

const frascoRef = admin.firestore().collection("Frasco_Reagente").doc(
  dados.idFrasco);
return admin.firestore().runTransaction(async (tx) => {
  const snap = await tx.get(frascoRef);
  if (!snap.exists) throw new HttpsError("not-found", "Frasco não
    encontrado.");
  const frasco = snap.data()!;
  await validarGestorDoAlmojarifado(request.auth!.uid, frasco.
    id_almojarifado);

  if (frasco.estado_fisico_frasco !== "FECHADO") {
    throw new HttpsError("failed-precondition", "Somente um frasco FECHADO
      pode ser aberto por esta operação.");
  }

  const agora = new Date();
  const validadeEfetiva = calcularValidadeEfetivaNaAbertura(frasco, agora);

  const venceu = Boolean(validadeEfetiva && validadeEfetiva <= agora);

  if (venceu && !dados.destinoSeVencerNaAbertura) {
    throw new HttpsError("failed-precondition", "A abertura tornou o
      frasco vencido. O gestor deve escolher o destino do frasco.");
  }
  if (venceu && dados.destinoSeVencerNaAbertura === "QUARENTENA" && !dados.
    detalheStatus) {
    throw new HttpsError("invalid-argument", "A entrada em quarentena
      exige detalhe_status.");
  }

  tx.update(frascoRef, {
    estado_fisico_frasco: "ABERTO",
    data_abertura: agora,
    validade_efetiva: validadeEfetiva,
    vencido: venceu,
    em_quarentena: venceu && dados.destinoSeVencerNaAbertura === "
      QUARENTENA",
```

```

        uso_vencido_authorized: venceu && dados.destinoSeVencerNaAbertura ===
        "DISPONIVEL",
        detalhe_status: venceu
        ? (dados.detalheStatus ?? (dados.destinoSeVencerNaAbertura === "
        PENDENTE_DE_DESCARTE"
        ? "Frasco vencido aguardando processo institucional de descarte."
        : null))
        : null,
    });

    return { validadeEfetiva, venceu, destino: dados.
    destinoSeVencerNaAbertura ?? null };
  });
});

```

Registrar Retirada (Empréstimo)

```

interface RetiradaFrasco {
  idFrasco: string;
  idUsuarioRetirou: string;
  idLocalUsado: string;
  pesoSaida: number;
  dataDevolucaoPrevista: string;
  confirmarUsoVencido?: boolean;
  abrirNoEmprestimo?: boolean;
  finalidadeUso: "PESQUISA" | "DIDATICO_DEMONSTRACAO" | "OUTRO";
}

export const registrarRetirada = onCall(async (request) => {
  const dados = request.data as RetiradaFrasco;
  validarPermissao(request, ["Chefe_Geral", "Gestor_Almoxxarifado"]);

  // Verificação do papel do tomador: este UID é de outro usuário
  // (não do requisitante), portanto Custom Claims não estão disponíveis.
  // Leitura Firestore é necessária aqui (1 leitura, não 6).
  const colecoesBorrower = ["Professor", "Bolsista"];
  const [profSnap, bolsSnap] = await Promise.all([
    admin.firestore().collection("Professor").doc(dados.idUsuarioRetirou).
    get(),
    admin.firestore().collection("Bolsista").doc(dados.idUsuarioRetirou).get
    (),
  ]);
});

```



```
if (!profSnap.exists && !bolsSnap.exists) {
  throw new HttpsError("failed-precondition", "Usuário selecionado não é
  Professor nem Bolsista.");
}

const frascoRef = admin.firestore().collection("Frasco_Reagente").doc(
  dados.idFrasco);

return admin.firestore().runTransaction(async (tx) => {
  const frascoSnap = await tx.get(frascoRef);
  if (!frascoSnap.exists) throw new HttpsError("not-found", "Frasco não
  encontrado.");
  const frasco = frascoSnap.data()!;

  await validarGestorDoAlmoxarifado(request.auth!.uid, frasco.
  id_almoxarifado);
  if (frasco.disponibilidade !== "DISPONIVEL") {
    throw new HttpsError("failed-precondition", "Frasco não está disponí
    vel para retirada.");
  }
  if (frasco.em_quarentena) {
    throw new HttpsError("failed-precondition", "Frasco em quarentena não
    pode ser retirado.");
  }

  const agora = new Date();
  let frascoVencido = frasco.vencido;
  let validadeEfetiva = frasco.validade_efetiva ? frasco.validade_efetiva.
  toDate() : null;

  if (dados.abrirNoEmprestimo && frasco.estado_fisico_frasco === "FECHADO"
  ) {
    validadeEfetiva = calcularValidadeEfetivaNaAbertura(frasco, agora);
    frascoVencido = Boolean(validadeEfetiva && validadeEfetiva <= agora);
    tx.update(frascoRef, {
      estado_fisico_frasco: "ABERTO",
      data_abertura: agora,
      validade_efetiva: validadeEfetiva,
      vencido: frascoVencido,
    });
  }
}
```

```
if (frascoVencido && !frasco.uso_vencido_autorizado && !dados.
confirmarUsoVencido) {
  throw new HttpsError("failed-precondition", "Frasco vencido requer
confirmação explícita de uso didático.");
}
// P9: finalidade_uso vem do frontend; backend impõe DIDATICO se vencido
const finalidade = frascoVencido ? "DIDATICO_DEMONSTRACAO" : dados.
finalidadeUso;

const emprestimoRef = admin.firestore().collection("Emprestimo_Reagente"
).doc();
tx.set(emprestimoRef, {
  id_frasco_reagente: dados.idFrasco,
  id_usuario_retirou: dados.idUsuarioRetirou,
  id_almoxarifado: frasco.id_almoxarifado,
  status: "EM_USO",
  data_retirada: admin.firestore.FieldValue.serverTimestamp(),
  data_devolucao_prevista: new Date(dados.dataDevolucaoPrevista),
  id_local_usado: dados.idLocalUsado,
  peso_saida: dados.pesoSaida,
  uso_vencido_aceito: Boolean(frascoVencido),
  finalidade_uso: finalidade,
});

tx.update(frascoRef, { disponibilidade: "EMPRESTADO" });

const historicoRef = admin.firestore().collection("
Historico_Frasco_Reagente").doc();
tx.set(historicoRef, {
  id_frasco_reagente: dados.idFrasco,
  id_almoxarifado: frasco.id_almoxarifado,
  id_gestor: request.auth!.uid,
  tipo: "SAIU",
  id_emprestimo_reagente: emprestimoRef.id,
  timestamp: admin.firestore.FieldValue.serverTimestamp(),
});

return { idEmprestimo: emprestimoRef.id };
});
});
```

Registrar Devolução com atualização física e histórico completo

```
interface DevolucaoFrasco {
  idEmprestimo: string;
  pesoRetorno: number;
  destinoPosDevolucao?: "QUARENTENA" | "PENDENTE_DE_DESCARTE" | "DISPONIVEL";
}

export const registrarDevolucao = onCall(async (request) => {
  const dados = request.data as DevolucaoFrasco;
  await validarPermissao(request.auth?.uid, ["Chefe_Geral", "Gestor_Almoxxarifado"]);

  const emprestimoRef = admin.firestore().collection("Emprestimo_Reagente").doc(dados.idEmprestimo);

  return admin.firestore().runTransaction(async (tx) => {
    const emprestimoSnap = await tx.get(emprestimoRef);
    if (!emprestimoSnap.exists) throw new HttpsError("not-found", "Empré stimo não encontrado.");
    const emprestimo = emprestimoSnap.data()!;
    if (emprestimo.status === "DEVOLVIDO" || emprestimo.status === "DEVOLVIDO_COM_ATRASO") {
      throw new HttpsError("failed-precondition", "Empréstimo já foi devolvido.");
    }

    const frascoRef = admin.firestore().collection("Frasco_Reagente").doc(emprestimo.id_frasco_reagente);
    const frascoSnap = await tx.get(frascoRef);
    const frasco = frascoSnap.data()!;
    await validarGestorDoAlmoxxarifado(request.auth!.uid, request.auth!.token, frasco.id_almoxxarifado);

    const densidade = await resolverDensidadeDoFrasco(frasco);
    const pesoConsumido = Math.max(0, emprestimo.peso_saida - dados.pesoRetorno);

    // C5: Reagentes higroscópicos (NaOH, s\`{i}lica gel, etc.) podem ganhar massa por
    // absorção de umidade. Permite margem de +2% da massa de saída antes de
```

```
erro.  
  
const MARGEM_HIGROSCOPICA = 0.02;  
const limiteMaxRetorno = emprestimo.peso_saida * (1 +  
MARGEM_HIGROSCOPICA);  
if (dados.pesoRetorno > limiteMaxRetorno) {  
  throw new HttpsError("invalid-argument",  
    'Peso de retorno (${dados.pesoRetorno}g) excede 102% da massa de saída  
da (${emprestimo.peso_saida}g). Verifique se há contaminação ou erro na  
balança.');}  
  
const avisoHigroscopico = dados.pesoRetorno > emprestimo.peso_saida;  
const volumeUtilizado = densidade ? pesoConsumido / densidade :  
pesoConsumido;  
  
const agora = new Date();  
// Normalização: Prazo encerra-se às 23:59:59.999 do dia previsto  
const prazoLimite = new Date(emprestimo.data_devolucao_prevista.toDate()  
);  
prazoLimite.setHours(23, 59, 59, 999);  
const atrasado = agora > prazoLimite;  
  
const validadeEfetiva = frasco.validade_efetiva?.toDate?.() ?? frasco.  
validade_efetiva ?? null;  
const venceuAgora = Boolean(validadeEfetiva && validadeEfetiva <= agora);  
  
const frascoVencido = Boolean(frasco.vencido || venceuAgora);  
  
const atualizacaoFrasco: Record<string, unknown> = {  
  disponibilidade: "DISPONIVEL",  
  data_ultima_pesagem: admin.firestore.FieldValue.serverTimestamp(),  
  peso_atual: dados.pesoRetorno,  
  medida_usada: admin.firestore.FieldValue.increment(volumeUtilizado),  
  vencido: frascoVencido,  
};  
  
if (frascoVencido && dados.destinoPosDevolucao === "QUARENTENA") {  
  atualizacaoFrasco.em_quarentena = true;  
  atualizacaoFrasco.uso_vencido_autorizado = false;  
  atualizacaoFrasco.detalhe_status = "Frasco vencido retido em  
quarentena após devolução."  
} else if (frascoVencido && dados.destinoPosDevolucao === "DISPONIVEL")
```

```
{
  atualizacaoFrasco.em_quarentena = false;
  atualizacaoFrasco.uso_vencido_autorizado = true;
}

tx.update(emprestimoRef, {
  status: atrasado ? "DEVOLVIDO_COM_ATRASO" : "DEVOLVIDO",
  data_devolucao_efetuada: admin.firestore.FieldValue.serverTimestamp(),
  id_usuario_devolveu: request.auth!.uid,
  peso_retorno: dados.pesoRetorno,
  medida_utilizada: volumeUtilizado,
  unidade_medida_utilizada: densidade ? "ml" : "g",
  ...(avisoHigroscopico && { aviso: "higroscopico_suspeito" }),
});
tx.update(frascoRef, atualizacaoFrasco);

// Registro obrigatório de devolução no histórico
const histRef = admin.firestore().collection("Historico_Frasco_Reagente")
.doc();
tx.set(histRef, {
  id_frasco_reagente: emprestimo.id_frasco_reagente,
  id_almoxarifado: frasco.id_almoxarifado,
  id_gestor: request.auth!.uid,
  tipo: "ENTROU",
  id_emprestimo_reagente: emprestimoRef.id,
  peso_anterior: emprestimo.peso_saida,
  peso_novo: dados.pesoRetorno,
  medida_ajustada: volumeUtilizado,
  unidade_medida_ajustada: densidade ? "ml" : "g",
  timestamp: admin.firestore.FieldValue.serverTimestamp(),
});

return { pesoConsumido, volumeUtilizado, atrasado, frascoVencido,
  avisoHigroscopico: avisoHigroscopico || false };
});
});
```

10.2.3 Validação síncrona de validade no cadastro e na abertura

A validade não depende apenas do job diário. No cadastro do frasco, a Cloud Function calcula *validade_efetiva* e compara esse valor com o relógio do servidor. Se a validade já

expirou, a criação só pode prosseguir com uma decisão explícita do gestor: QUARENTENA, PENDENTE_DE_DESCARTE ou DISPONIVEL com *uso_vencido_autorizado*. A primeira opção exige *detalhe_status*. A terceira representa uma exceção operacional deliberada: reagentes vencidos podem continuar sendo utilizados em práticas menos exigentes ou demonstrações quando o gestor registra explicitamente essa autorização.

Na abertura do frasco, a função *calcularValidadeEfetivaNaAbertura* aplica:

$$validade_efetiva = \min(validade_fechado, data_abertura + validade_apos_aberto_dias),$$

quando os dois limites existem.

10.2.4 Jobs Agendados: Vencimento, Atraso e Materialização

Vencimento e Atraso com busca preventiva de devoluções

```
import { onSchedule } from "firebase-functions/v2/scheduler";
import { onDocumentCreated, onDocumentDeleted, onDocumentUpdated } from "
  firebase-functions/v2/firestore";

export const verificarVencimentosEAtrasos = onSchedule("every day 03:00",
  async () => {
    const agora = new Date();
    const inicioHoje = inicioDoDiaInstitucional(agora);
    const fimDeAmanha = fimDoDiaInstitucional(adicionarDias(inicioHoje, 1));

    const emprestimosNaJanela = await admin.firestore().collection("
      Empréstimo_Reagente")
      .where("status", "==", "EM_USO")
      .where("data_devolucao_prevista", "<=", fimDeAmanha)
      .get();

    const batchAtrasados = admin.firestore().batch();
    const notificacoesPreventivas: Array<{ idEmpréstimo: string; idUsuario:
      string; quando: "HOJE" | "AMANHÃ" }> = [];

    emprestimosNaJanela.docs.forEach((doc) => {
      const dados = doc.data();
      const prevista = dados.data_devolucao_prevista.toDate();

      if (prevista < inicioHoje) {
        batchAtrasados.update(doc.ref, { status: "ATRASADO" });
        return;
      }
    });
  });
```

```
}

if (mesmoDia(prevista, inicioHoje)) {
  notificacoesPreventivas.push({ idEmprestimo: doc.id, idUsuario: dados.
id_usuario_retirou, quando: "HOJE" });
} else {
  notificacoesPreventivas.push({ idEmprestimo: doc.id, idUsuario: dados.
id_usuario_retirou, quando: "AMANHA" });
}
});

await batchAtrasados.commit();

for (const item of notificacoesPreventivas) {
  await notificarProfessorDevolucaoUmaVez(item.idUsuario, item.
idEmprestimo, item.quando);
}

const totalAtrasados = empréstimosNaJanela.docs.filter((doc) => {
  const prevista = doc.data().data_devolucao_prevista.toDate();
  return prevista < inicioHoje;
}).length;
if (totalAtrasados > 0) {
  await notificarGestoresDeReagentes("ENTREGA_ATRASADA", totalAtrasados);
}

const frascosVencendo = await admin.firestore().collection("
Frasco_Reagente")
  .where("vencido", "==", false)
  .where("validade_efetiva", "<=", agora)
  .get();

const batchVencidos = admin.firestore().batch();
frascosVencendo.docs.forEach((doc) => {
  batchVencidos.update(doc.ref, { vencido: true });
  const histRef = admin.firestore().collection("Historico_Frasco_Reagente")
).doc();
  batchVencidos.set(histRef, {
    id_frasco_reagente: doc.id,
    id_almoxarifado: doc.data().id_almoxarifado,
    tipo: "VENCEU",
```

```
        timestamp: admin.firestore.FieldValue.serverTimestamp(),
    });
});
await batchVencidos.commit();
if (!frascosVencendo.empty) {
    await notificarGestoresDeReagentes("FRASCOS_VENCIDOS", frascosVencendo.size);
}
});

function adicionarDias(data: Date, dias: number): Date {
    const result = new Date(data);
    result.setDate(result.getDate() + dias);
    return result;
}

function mesmoDia(a: Date, b: Date): boolean {
    return a.getFullYear() === b.getFullYear() &&
        a.getMonth() === b.getMonth() &&
        a.getDate() === b.getDate();
}

function inicioDoDiaInstitucional(data: Date): Date {
    const result = new Date(data);
    result.setHours(0, 0, 0, 0);
    return result;
}

function fimDoDiaInstitucional(data: Date): Date {
    const result = new Date(data);
    result.setHours(23, 59, 59, 999);
    return result;
}

export const onFrascoCriado = onDocumentCreated("Frasco_Reagente/{frascoId}",
    async (event) => {
        const idLote = event.data?.data().id_lote;
        if (idLote) await atualizarContadorLote(idLote, +1);
    });
```



```
export const onFrascoRemovido = onDocumentDeleted("Frasco_Reagente/{frascoId}", async (event) => {
  const idLote = event.data?.data().id_lote;
  if (idLote) await atualizarContadorLote(idLote, -1);
});

async function atualizarContadorLote(idLote: string, delta: number) {
  await admin.firestore().collection("Lote_Materializado").doc(idLote).set(
    { id_lote: idLote, qtd_frascos_cadastrados: admin.firestore.FieldValue.increment(delta) },
    { merge: true }
  );
}
```

10.2.5 Fluxo de Bens Patrimoniais: Requisições

Criação e Resposta de Requisição de Edição (C2: lock determinístico)

```
export const criarRequisicaoEdicaoBem = onCall(async (request) => {
  validarPermissao(request, ["Professor"]);
  const dados = request.data as { idBemPatrimonial: string; novoNome?: string; novoStatus?: string; novoEstadoConservacao?: string; novoIdLocal?: string; motivo: string };

  // C2: Lock determinístico - elimina phantom reads em transações Firestore.

  // Se dois professores enviarem simultaneamente, apenas um cria o lock; o outro
  // recebe ALREADY_EXISTS da transação e falha antes de criar a requisição.
  const lockId = `bem_edicao_${dados.idBemPatrimonial}`;
  const lockRef = admin.firestore().collection("Locks_Requisicao_Patrimonio").doc(lockId);
  const reqRef = admin.firestore().collection("Requisicao_Edicao_Bem_Patrimonial").doc();

  return admin.firestore().runTransaction(async (tx) => {
    const lockSnap = await tx.get(lockRef);
    if (lockSnap.exists) {
      throw new HttpsError("failed-precondition", "Já existe uma requisição de edição pendente para este bem.");
    }
  })
}
```

```
// Cria lock e requisição atomicamente
tx.set(lockRef, { criado_em: admin.firestore.FieldValue.serverTimestamp
() });
tx.set(reqRef, {
  id_bem_patrimonial: dados.idBemPatrimonial,
  novo_nome: dados.novoNome ?? null,
  novo_status: dados.novoStatus ?? null,
  novo_estado_conservacao: dados.novoEstadoConservacao ?? null,
  novo_id_local: dados.novoIdLocal ?? null,
  motivo: dados.motivo,
  status: "pendente",
  feita_em: admin.firestore.FieldValue.serverTimestamp(),
  id_usuario_solicitante: request.auth!.uid,
});
return { idRequisicao: reqRef.id };
});
});

export const responderRequisicaoEdicaoBem = onCall(async (request) => {
  validarPermissao(request, ["Chefe_Geral", "Gestor_Bens_Patrimoniais"]);
  const { idRequisicao, aprovar, justificativa } = request.data as {
    idRequisicao: string; aprovar: boolean; justificativa: string };
  const reqRef = admin.firestore().collection("
    Requisicao_Edicao_Bem_Patrimonial").doc(idRequisicao);

  return admin.firestore().runTransaction(async (tx) => {
    const reqSnap = await tx.get(reqRef);
    if (!reqSnap.exists) throw new HttpsError("not-found", "Requisição não
    encontrada.");
    const req = reqSnap.data()!;
    if (req.status !== "pendente") {
      throw new HttpsError("failed-precondition", "Requisição já foi
      respondida.");
    }

    // Remove o lock ao responder
    const lockRef = admin.firestore().collection("
    Locks_Requisicao_Patrimonio")
      .doc(`bem_edicao_${req.id_bem_patrimonial}`);
    tx.delete(lockRef);
```

```

tx.update(reqRef, {
  status: aprovar ? "aprovada" : "rejeitada",
  respondida_em: admin.firestore.FieldValue.serverTimestamp(),
  id_usuario_respondente: request.auth!.uid,
  justificativa_resposta: justificativa,
});

if (aprovar) {
  const bemRef = admin.firestore().collection("Bem_Patrimonial").doc(req.
id_bem_patrimonial);
  const bemSnap = await tx.get(bemRef);
  if (!bemSnap.exists) throw new HttpsError("not-found", "Bem
patrimonial não encontrado.");
  const bem = bemSnap.data()!;

  const camposBem: Record<string, unknown> = {};
  if (req.novo_status) camposBem.status = req.novo_status;
  if (req.novo_estado_conservacao) camposBem.estado_conservacao = req.
novo_estado_conservacao;
  if (req.novo_id_local) camposBem.id_local = req.novo_id_local;

  if (req.novo_nome) {
    // Reclassificação de modelo sem afetar outros bens
    const resumoRef = admin.firestore().collection("
Resumo_Bem_Patrimonial").doc(bem.id_resumo_bem_patrimonial);
    camposBem.nome_equipamento = req.novo_nome;
  }

  tx.update(bemRef, camposBem);
}
return { status: aprovar ? "aprovada" : "rejeitada" };
});
});

```

criarRequisicaoAdicaoBem com lock determinístico (C2)

```

export const criarRequisicaoAdicaoBem = onCall(async (request) => {
  validarPermissao(request, ["Professor"]);

  const dados = request.data as {
    numeroPatrimonioProposto: string;
    estadoConservacaoProposto: string;
  };

```

```
idLocal: string;
nomeResponsavelProposto: string;
idResumoBemPatrimonial?: string;
nomeResumoProposto?: string;
descricaoResumoProposta?: string;
motivo: string;
};

// C2: Lock determinístico por número de patrimônio proposto
const lockId = `bem_adicao_${dados.numeroPatrimonioProposto}`;
const lockRef = admin.firestore().collection("Locks_Requisicao_Patrimonio")
  .doc(lockId);
const reqRef = admin.firestore().collection("Requisicao_Adicao_Bem_Patrimonial").doc();

return admin.firestore().runTransaction(async (tx) => {
  const lockSnap = await tx.get(lockRef);
  if (lockSnap.exists) {
    throw new HttpsError("failed-precondition", "Já existe requisição
    pendente para este número de patrimônio.");
  }
  tx.set(lockRef, { criado_em: admin.firestore.FieldValue.serverTimestamp() });
  tx.set(reqRef, {
    numero_patrimonio_proposto: dados.numeroPatrimonioProposto,
    estado_conservacao_proposto: dados.estadoConservacaoProposto,
    photo_url_proposta: null,
    nome_responsavel_proposto: dados.nomeResponsavelProposto,
    id_local: dados.idLocal,
    id_resumo_bem_patrimonial: dados.idResumoBemPatrimonial ?? null,
    nome_resumo_proposto: dados.nomeResumoProposto ?? null,
    descricao_resumo_proposta: dados.descricaoResumoProposta ?? null,
    motivo: dados.motivo,
    status: "pendente",
    feita_em: admin.firestore.FieldValue.serverTimestamp(),
    id_usuario_solicitante: request.auth!.uid,
    respondida_em: null,
    id_usuario_respondente: null,
  });
  return { idRequisicao: reqRef.id };
});
```

```
});

export const responderRequisicaoAdicaoBem = onCall(async (request) => {
  validarPermissao(request, ["Chefe_Geral", "Gestor_Bens_Patrimoniais"]);
  const { idRequisicao, aprovar, justificativa } = request.data as {
    idRequisicao: string; aprovar: boolean; justificativa: string };
  const reqRef = admin.firestore().collection("
    Requisicao_Adicao_Bem_Patrimonial").doc(idRequisicao);

  return admin.firestore().runTransaction(async (tx) => {
    const reqSnap = await tx.get(reqRef);
    if (!reqSnap.exists) throw new HttpsError("not-found", "Requisição não
    encontrada.");
    const req = reqSnap.data()!;
    if (req.status !== "pendente") throw new HttpsError("failed-precondition
    ", "Requisição já respondida.");

    // C2: Remove o lock ao responder (libera possível nova requisição
    futura)
    const lockRef = admin.firestore().collection("
    Locks_Requisicao_Patrimonio")
      .doc(`bem_adicao_${req.numero_patrimonio_proposto}`);
    tx.delete(lockRef);

    let idBemCriado: string | null = null;

    if (aprovar) {
      let idResumo = req.id_resumo_bem_patrimonial;

      // Se não havia resumo existente, cria atômica e transacionalmente o
      Resumo_Bem_Patrimonial
      if (!idResumo && req.nome_resumo_proposto) {
        const novoResumoRef = admin.firestore().collection("
        Resumo_Bem_Patrimonial").doc();
        tx.set(novoResumoRef, {
          nome: req.nome_resumo_proposto,
          descricao: req.descricao_resumo_proposta ?? "",
          criado_em: admin.firestore.FieldValue.serverTimestamp(),
        });
        idResumo = novoResumoRef.id;
      }
    }
  });
});
```

```
    if (!idResumo) throw new HttpsError("failed-precondition", "A aprovação exige vincular um resumo.");

    const bemRef = admin.firestore().collection("Bem_Patrimonial").doc();
    idBemCriado = bemRef.id;

    tx.set(bemRef, {
      id_resumo_bem_patrimonial: idResumo,
      nome_equipamento: req.nome_resumo_proposto ?? "Equipamento",
      numero_patrimonio: req.numero_patrimonio_proposto,
      estado_conservacao: req.estado_conservacao_proposto,
      id_local: req.id_local,
      photo_url: req.photo_url_proposta ?? "https://placeholder",
      documento_dado_baixa_pdf_url: null,
      nome_responsavel_sei: req.nome_responsavel_proposto,
      status: "Ativo",
      cadastrado_em: admin.firestore.FieldValue.serverTimestamp(),
    });

    tx.update(reqRef, {
      status: "aprovada",
      respondida_em: admin.firestore.FieldValue.serverTimestamp(),
      id_usuario_respondente: request.auth!.uid,
      id_bem_patrimonial_se_aprovado: idBemCriado,
      justificativa_resposta: justificativa,
    });
  } else {
    tx.update(reqRef, {
      status: "rejeitada",
      respondida_em: admin.firestore.FieldValue.serverTimestamp(),
      id_usuario_respondente: request.auth!.uid,
      justificativa_resposta: justificativa,
    });
  }

  return { status: aprovar ? "aprovada" : "rejeitada", idBemCriado };
});
});

export const onResumoBemPatrimonialNomeAtualizado = onDocumentUpdated(
```

```

"Resumo_Bem_Patrimonial/{resumoId}",
async (event) => {
  const antes = event.data?.before.data();
  const depois = event.data?.after.data();
  if (!antes || !depois || antes.nome === depois.nome) return;

  const bens = await admin.firestore().collection("Bem_Patrimonial")
    .where("id_resumo_bem_patrimonial", "==", event.params.resumoId)
    .get();

  const chunks = [];
  for (let i = 0; i < bens.docs.length; i += 400) {
    chunks.push(bens.docs.slice(i, i + 400));
  }
  for (const chunk of chunks) {
    const batch = admin.firestore().batch();
    chunk.forEach((bem) => batch.update(bem.ref, { nome_equipamento:
depois.nome }));
    await batch.commit();
  }
}
);

```

P13: Propagação de Local para Bem_Patrimonial

```

export const onLocalAtualizado = onDocumentUpdated(
  "Local/{localId}",
  async (event) => {
    const antes = event.data?.before.data();
    const depois = event.data?.after.data();
    if (!antes || !depois) return;
    if (antes.predio === depois.predio && antes.andar === depois.andar &&
antes.sala === depois.sala) return;

    const bens = await admin.firestore().collection("Bem_Patrimonial")
      .where("id_local", "==", event.params.localId)
      .get();

    const chunks = [];
    for (let i = 0; i < bens.docs.length; i += 400) {
      chunks.push(bens.docs.slice(i, i + 400));
    }
  }
);

```

```
for (const chunk of chunks) {
  const batch = admin.firestore().batch();
  chunk.forEach((bem) => batch.update(bem.ref, {
    predio: depois.predio,
    andar: depois.andar,
    sala: depois.sala,
  }));
  await batch.commit();
}
};
```

P14: Validação de re-entrada de aluno por código

```
export const ingressarEmTurmaPorCodigo = onCall(async (request) => {
  validarPermissao(request, ["Aluno"]);
  const { codigoTurma } = request.data as { codigoTurma: string };

  const turmaSnap = await admin.firestore().collection("Turma")
    .where("codigo_turma", "==", codigoTurma)
    .where("status", "==", "Ativo")
    .limit(1).get();
  if (turmaSnap.empty) throw new HttpsError("not-found", "Turma não encontrada ou arquivada.");
  const turmaDoc = turmaSnap.docs[0];
  const turma = turmaDoc.data();

  return admin.firestore().runTransaction(async (tx) => {
    // Checar capacidade
    const alunosSnap = await tx.get(
      turmaDoc.ref.collection("Alunos")
    );
    if (alunosSnap.size >= turma.capacidade) {
      throw new HttpsError("failed-precondition", "Turma lotada.");
    }

    // Checar se já está matriculado
    const jaMatriculado = alunosSnap.docs.some(
      (d) => d.id === request.auth!.uid
    );
    if (jaMatriculado) {
      throw new HttpsError("already-exists", "Você já está nesta turma.");
    }
  });
});
```



```

    }

    // P14: Checar se foi removido anteriormente
    const historico = await tx.get(
      turmaDoc.ref.collection("HistoricoAlunos")
        .where("id_aluno", "==", request.auth!.uid)
        .where("tipo", "==", "exclusao_aluno")
        .limit(1)
    );
    if (!historico.empty) {
      throw new HttpsError("failed-precondition",
        "Você foi removido desta turma pelo professor. O re-ingresso requer convite explícito.");
    }

    // Matricular
    tx.set(turmaDoc.ref.collection("Alunos").doc(request.auth!.uid), {
      id_aluno: request.auth!.uid,
      ingressou_em: admin.firestore.FieldValue.serverTimestamp(),
    });
    tx.set(turmaDoc.ref.collection("HistoricoAlunos").doc(), {
      id_aluno: request.auth!.uid,
      tipo: "inclusao_aluno",
      timestamp: admin.firestore.FieldValue.serverTimestamp(),
    });

    return { idTurma: turmaDoc.id, nomeTurma: turma.nome_turma };
  });
});

```

10.2.6 Relatórios em PDF

Pseudo-códigos das funções de relatório, revisados: os nomes de campo passam a corresponder exatamente ao modelo (*data_devolucao_efetuada*, não *data_devolucao*; filtro por *id_almojarifado* denormalizado em *Emprestimo_Reagente*, não por um inexistente *id_local_saida*), e *gerarRelatorioAlmojarifado* passa a checar o escopo do gestor, não apenas o papel.

Configuração Base e Relatório de Almojarifado

```
import PDFDocument from "pdfkit-table";
```

```
interface FiltrosAlmoxarifado {
  idAlmoxarifado: string;
  mes: number;
  ano: number;
}

export const gerarRelatorioAlmoxarifado = onCall(async (request) => {
  const { idAlmoxarifado, mes, ano } = request.data as FiltrosAlmoxarifado;
  validarPermissao(request, ["Chefe_Geral", "Gestor_Almoxarifado"]);
  await validarGestorDoAlmoxarifado(request.auth!.uid, request.auth!.token,
    idAlmoxarifado);

  const dataInicio = new Date(ano, mes - 1, 1);
  const dataFim = new Date(ano, mes, 0);

  const devolucoesSnapshot = await admin.firestore().collection("
    Emprestimo_Reagente")
    .where("id_almoxarifado", "==", idAlmoxarifado)
    .where("data_devolucao_efetuada", ">=", dataInicio)
    .where("data_devolucao_efetuada", "<=", dataFim)
    .get();

  const volumeTotalConsumido = devolucoesSnapshot.docs.reduce(
    (acc, doc) => acc + (doc.data().medida_utilizada || 0), 0
  );

  const historicoSnapshot = await admin.firestore().collection("
    Historico_Frasco_Reagente")
    .where("id_almoxarifado", "==", idAlmoxarifado)
    .where("timestamp", ">=", dataInicio)
    .where("timestamp", "<=", dataFim)
    .orderBy("timestamp", "asc").get();

  const doc = new PDFDocument({ margin: 30, size: "A4" });
  const buffer = await buildPdfBuffer(doc, historicoSnapshot.docs,
    volumeTotalConsumido);

  const fileName = `relatorios/almoxarifado_${idAlmoxarifado}_${mes}_${ano}
    _${Date.now()}.pdf`;
  const fileRef = admin.storage().bucket().file(fileName);
  await fileRef.save(buffer, { contentType: "application/pdf" });
});
```

```
const [url] = await fileRef.getSignedUrl({ action: "read", expires: Date.  
  now() + 3600000 });  
return { url };  
});
```

Relatório Mensal de um Prédio (Filtros Compostos)

```
interface FiltrosPredio {  
  predio: string;  
  mes: number;  
  ano: number;  
  andar?: string;  
  sala?: string;  
  estadoConservacao?: string;  
  status?: string;  
}  
  
export const gerarRelatorioBensPredio = onCall(async (request) => {  
  const filtros = request.data as FiltrosPredio;  
  validarPermissao(request, ["Chefe_Geral", "Gestor_Bens_Patrimoniais"]);  
  
  let query = admin.firestore().collection("Bem_Patrimonial").where("predio",  
    "==", filtros.predio);  
  if (filtros.andar) query = query.where("andar", "==", filtros.andar);  
  if (filtros.sala) query = query.where("sala", "==", filtros.sala);  
  if (filtros.estadoConservacao) query = query.where("estado_conservacao", "  
    ==", filtros.estadoConservacao);  
  if (filtros.status) query = query.where("status", "==", filtros.status);  
  
  const bensSnapshot = await query.get();  
  
  const doc = new PDFDocument({ size: "A4" });  
  const buffer = await buildPredioPdfBuffer(doc, bensSnapshot.docs, filtros);  
  
  const fileRef = admin.storage().bucket().file(`relatorios/predio_${filtros.  
    predio}_${Date.now()}.pdf`);  
  await fileRef.save(buffer, { contentType: "application/pdf" });  
  
  const [url] = await fileRef.getSignedUrl({ action: "read", expires: Date.  
    now() + 3600000 });
```

```
    return { url };  
  });
```

Relatório Geral e Período Personalizado

```
interface FiltrosGeralEPersonalizado {  
  dataInicio: string;  
  dataFim: string;  
  entidade: "Bens_Patrimoniais" | "Reagentes";  
  prediosSelecionados?: string[];  
}  
  
export const gerarRelatorioPersonalizado = onCall(async (request) => {  
  const { dataInicio, dataFim, entidade, prediosSelecionados } = request.  
    data as FiltrosGeralEPersonalizado;  
  validarPermissao(request, ["Chefe_Geral", "Gestor_Bens_Patrimoniais", "  
    Gestor_Almoxxarifado"]);  
  
  const dataIniObj = new Date(dataInicio);  
  const dataFimObj = new Date(dataFim);  
  let snapshot;  
  
  if (entidade === "Bens_Patrimoniais") {  
    let query = admin.firestore().collection("Historico_Bem_Patrimonial")  
      .where("timestamp", ">=", dataIniObj)  
      .where("timestamp", "<=", dataFimObj);  
    if (prediosSelecionados && prediosSelecionados.length > 0) {  
      query = query.where("predio", "in", prediosSelecionados);  
    }  
    snapshot = await query.get();  
  } else {  
    snapshot = await admin.firestore().collection("Emprestimo_Reagente")  
      .where("data_devolucao_efetuada", ">=", dataIniObj)  
      .where("data_devolucao_efetuada", "<=", dataFimObj)  
      .get();  
  }  
  
  const doc = new PDFDocument({ layout: "landscape", size: "A4" });  
  const buffer = await buildPersonalizadoPdfBuffer(doc, snapshot.docs,  
    entidade);  
  
  const fileRef = admin.storage().bucket().file('relatorios/personalizado_${
```

```
    Date.now()}.pdf');  
    await fileRef.save(buffer, { contentType: "application/pdf" });  
    const [url] = await fileRef.getSignedUrl({ action: "read", expires: Date.  
        now() + 3600000 });  
    return { url };  
});
```

11 Regras de Segurança do Firestore (Security Rules)

i Por que as Security Rules são obrigatórias

Toda leitura via `onSnapshot` no frontend passa **diretamente** pelo Firestore SDK, **sem** passar pelas Cloud Functions. Portanto, as Cloud Functions são a camada de autorização para escrita, mas as *Security Rules* são a camada de autorização para **leitura direta**. Sem rules corretas, qualquer usuário autenticado poderia ler qualquer coleção.

Princípio: Deny-All por Padrão

A regra raiz do arquivo `firestore.rules` deve ser:

```
allow read, write: if false;
```

Todas as exceções são declaradas explicitamente abaixo.

11.1 Regras por Coleção

Coleção	Regra
Usuarios/{uid}	Read: <code>request.auth.uid == uid</code> (usuário lê só o próprio documento). Write: negado ao cliente (só Cloud Function).
Chefe_Geral, Gestor_Almojarifado, etc. (coleções de papel)	Read: <code>request.auth.uid == docId</code> (cada usuário lê só o próprio documento de papel). Write: negado ao cliente.
Turma/{turmaId}	Read: usuário é professor da turma (<code>resource.data.id_professor == request.auth.uid</code>) OU está na sub-coleção Alunos. Write: negado ao cliente.
Turma/{turmaId}/Alunos	Read: <code>request.auth.uid == resource.data.id_aluno</code> OU professor da turma.
Turma/{turmaId}/Posts	Read: membro da turma (professor ou aluno matriculado). Write: negado.
Turma/{turmaId}/Posts/{postId}/Comentarios	Read: <code>request.auth.uid == postId</code> (usuário lê só o próprio comentário).
Usuarios/{uid}/Notificacoes	Read: <code>request.auth.uid == uid</code> . Write: negado ao cliente.
Frasco_Reagente	Read: papel roles no token contém <code>Chefe_Geral</code> OU <code>Gestor_Almojarifado</code> OU <code>Professor</code> OU <code>Bolsista</code> OU <code>Aluno</code> . Write: negado ao cliente.

Coleção	Regra
Emprestimo_Reagente	Read: Chefe_Geral OU Gestor_Almoxxarifado OU o próprio usuário que retirou (<code>resource.data.id_usuario_retirou == request.auth.uid</code>). Write: negado.
Bem_Patrimonial, Almoxxarifado, Resumo_Reagente, Especificacao_Reagente, Substancia_Quimica, Materia, Resumo_Bem_Patrimonial	Read: aberto a qualquer usuário logado (<code>request.auth != null</code>) para preenchimento de telas. Write: negado ao cliente.
Requisicao_Edicao_Bem_Patrimonial	Read: Proprietario solicitante (<code>resource.data.id_usuario_solicitante == uid</code>) OU gestores de bens (papel no token). Write: negado.
Requisicao_Adicao_Bem_Patrimonial	Read: Proprietario .
Locks_Requisicao_Patrimonial	Read/Write: negado ao cliente (exclusivo de Cloud Function).
Registro_de_Auditoria	Read: Chefe_Geral no token. Write: negado ao cliente.
Roteiro_Experimento	Read: <code>resource.data.id_professor == uid</code> OU existe documento em Roteiro_Professor_Compartilhado com <code>id_professor_destino == uid</code> . Write: negado.

Leitura de Custom Claims nas Security Rules

O papel do usuário fica disponível nas Security Rules como `request.auth.token.roles`. Exemplo:

```
function temPapel(papel) {
  return papel in request.auth.token.roles;
}
```

Isso garante consistência com o mesmo Custom Claim usado pelas Cloud Functions (`resolverPapeisDoToken`).

12 Implementações em Estudo para Versões Futuras

i Escopo desta seção

Nada aqui faz parte do modelo 3FN das seções 4 e 5, nem deve ser implementado nesta primeira versão. É o registro de uma proposta para quando reagentes gasosos entrarem no almoxarifado do LCQUI – hoje nenhum reagente gasoso está cadastrado, por isso **GASOSO** foi removido de *Especificacao_Reagente.estado_fisico* (seção 4.17) nesta versão, em vez de deixado junto de campos pensados só para sólido/líquido.

12.1 Por que **GASOSO** sozinho não basta

Simplesmente devolver **GASOSO** a *estado_fisico* resolveria a classificação básica, mas um cilindro de gás carrega informação que **não** é estado físico e que não deve virar uma explosão de valores dentro do mesmo enum (**GASOSO_COMPRIMIDO**, **GASOSO_LIQUEFEITO**, **MISTURA_GASOSA_CALIBRACAO**, ...). São quatro dimensões independentes, que devem continuar independentes no banco:

Dimensão	Exemplos de valor
Estado físico	sólido, líquido, gasoso
Forma de armazenamento	gás comprimido, gás liquefeito, gás criogênico
Tipo de substância (já existe)	pura, mistura
Finalidade/aplicação	reagente comum, gás de calibração, gás de processo

Um cilindro de CO₂ puro, por exemplo, é *estado_fisico* = gasoso, *forma_armazenamento* = liquefeito (o CO₂ é mantido líquido sob pressão dentro do cilindro, embora seja gasoso em condição ambiente), *tipo_substancia* = pura. Já uma mistura de calibração (ex.: 500 ppm de CO₂ balanceado em N₂) é *estado_fisico* = gasoso, *forma_armazenamento* = comprimido, *tipo_substancia* = mistura, *finalidade* = calibração. Tratar “mistura gasosa de calibração” como uma categoria física própria misturaria uma propriedade química (o que a substância é) com uma propriedade comercial/de uso (para que ela serve) – o mesmo tipo de acoplamento indevido que a decomposição do *status* de *Frasco_Reagente* (seção 4.19) já evitou para outro campo.

12.2 Proposta de campos (apenas ilustrativa)

Especificacao_Reagente (campos adicionais propostos)

forma_armazenamento

ENUM NULL

CONVENCIONAL, GAS_COMPRIMIDO, GAS_LIQUEFEITO,
GAS_CRIOGENICO

finalidade

ENUM NULL

REAGENTE, CALIBRACAO, PROCESSO, OUTRO

Ambos ficam em *Especificacao_Reagente* – não em *Resumo_Reagente* nem em *Substancia_Quimica* – pelo mesmo motivo que *estado_fisico* e *unidade_de_medida* já vivem lá: são propriedades do produto comercial específico, não da substância química abstrata nem do item de catálogo. NULL para itens sólidos/líquidos, onde a dimensão não se aplica.

12.3 Além do enum: medição de gases não é pesagem

A Regra de Cálculo de Volume e Peso (seção 6.2.13) assume que o volume é sempre derivado de uma leitura de balança convertida por densidade. Para cilindros de gás, a prática de laboratório mais comum é ler a **pressão** no manômetro do próprio cilindro (ainda que pesar o cilindro também seja uma alternativa viável em alguns casos). Isso implica que *Frasco_Reagente* precisaria, no mínimo, de:

- *pressao_nominal_bar* – pressão de enchimento de fábrica.
- *pressao_atual_bar* – última leitura do manômetro (substituindo/complementando *peso_no_cadastrado/medida_usada* para este tipo de frasco).
- *capacidade_litros_cntp* – volume de gás equivalente em CNTP (Condições Normais de Temperatura e Pressão) quando o cilindro está cheio.
- *tipo_valvula* – necessário para compatibilidade de equipamento na retirada.

O cálculo de consumo também muda: em vez de $V = \Delta m / \rho$, a estimativa correta usa a proporcionalidade entre pressão e quantidade de gás restante (lei dos gases ideais, $PV = nRT$, ou uma correlação real para gases não-ideais em alta pressão), não uma conversão por densidade. Por isso este documento não tenta encaixar gases nos campos de peso já existentes: a reintrodução de **GASOSO** deve vir acompanhada desses campos e dessa fórmula alternativa, não apenas do valor no enum.

12.4 Horários de Turma

A entidade *Horario_Turma* (dia da semana, hora de início/fim e local por turma) foi removida da V1 por não possuir fluxo de uso definido nas telas nem nas regras de negócio. Em versões futuras, ela pode ser reintroduzida para permitir que o sistema exiba grades horárias, detecte conflitos de sala e envie lembretes automáticos antes do início das aulas.